02

IT Architecture in Practice

Book 2 Design evaluation and decision making

Prepared for Clifford Valiente

Vendor neutral concepts with Azure and AWS examples

16 chapters 54 exercises Four design studios

An eight week route with experiments and architecture dossiers

Second edition review edition
October 2026

Contents

How to use the design studio 3
01 Architecture as a defensible argument 4
02 Research experiments and architectural evidence 6
03 Integration studio brief and boundary design 8
04 Integration contracts and competing designs 10
05 Integration evaluation and a changed quota 13
06 Transaction studio brief and business invariants 15
07 Reservations payments and coordination 17
08 Transaction experiments and failure review 21
09 Data studio brief and semantic contracts 23
10 Pipeline alternatives and versioned publication 25
11 Data quality security and analytical evaluation 27
12 Modernisation studio brief and discovery 30
13 Data movement write fencing and recovery 32
14 Cutover rehearsal and modernisation review 34
15 Architecture review communication and judgement 36
16 Portfolio assessment and professional transfer 38
Exercise answers and assessment guidance 40
Experiments templates and research tasks 45
References 46

How to use the design studio

This second book turns architectural knowledge into a portfolio of argued decisions. It assumes the foundations in Book 1 and your existing programming and information-systems education. You will compare feasible designs, specify their guarantees, investigate failure, and revise a recommendation when evidence changes. The intended level is advanced undergraduate to postgraduate professional study. This is an original teaching course, not an accredited university module or an independently peer-reviewed textbook.

The four studios extend Meridian Service Portal, Aster Retail, Beacon Analytics and Orion Modernisation. Each studio explicitly supplies new fictional constraints. These override Book 1 only within that assignment; they do not revise the original cases. No workload, price, incident or experiment result in a fictional case is an industry benchmark.

Read the assignment brief and produce a first design before studying its worked analysis. Preserve that first version. Comparing your revisions teaches more than reproducing a polished answer. The worked designs demonstrate defensible reasoning; an alternative earns equal credit when it protects the stated invariants and explains cost, operation and uncertainty.

Outcomes and evidence

By the end, you should be able to produce an architecture dossier whose requirements, diagrams, contracts, failure behaviour and evidence agree; evaluate at least two credible alternatives; design a bounded experiment; interpret results without overstating their reach; and defend a conditional recommendation to technical and business audiences.

You will create four dossiers, four short research critiques, four experiment records and a comparative portfolio reflection. Keep evidence identifiers stable so that a reviewer can follow a requirement through a decision to a test. The supplied support files provide editable templates and local Python teaching experiments. None requires a paid cloud account.

An eight week study route

Plan five study days per week at four to five hours per day, approximately 160–200 hours. Reading is only part of this estimate. Allocate about a quarter of each day to reading, half to modelling or experiments, and the remainder to critique and revision. Revisit difficult foundations as needed.

Week	Reading and focus	Main output
1	Chapters 1 and 2	Evidence method and first experiment
2	Chapters 3 to 5	Integration dossier and recovery analysis
3	Chapters 6 to 8	Transaction design and failure argument
4	Chapters 9 to 11	Data contract and publication design
5	Chapters 12 to 14	Migration dossier and cutover exercise
6	Cross studio revisions	Alternative designs and research critiques
7	Chapter 15	Review meeting and revised decisions
8	Chapter 16	Final portfolio and oral defence

Source discipline

IEEE references are numbered separately within this book, in first-citation order. Foundational research supports a limited theoretical statement; official documentation supports a stated implementation property; the case analyses are original hypothetical reasoning. Product capabilities still need validation for the exact service, version, region and configuration before real implementation. Read the cited source when preparing a research critique, and distinguish its findings from your own application.

All figures are original teaching diagrams. A boundary means only what its caption and surrounding explanation define. A box is not proof of deployed redundancy, and an arrow is not proof of a transactional guarantee. Editable SVG figures are included with the authoring materials.

Second edition study and evidence route

Read the new advanced sections alongside their host chapters, then attempt exercises numbered .4 and .5 before consulting the advanced answers. Allow an additional estimated 12 to 18 hours per book for the new source reading, experiments and design comparisons. This extends the original study route; it is not an empirically measured completion time. The advanced practice folder contains local teaching programs with stated limits.

This review edition has an accompanying claim-to-source record, change record and external-review package. Full-text verification means checking the passages supporting a specified claim in an accessible full source; it does not imply that every sentence in every cited publication has been audited. Three enterprise references remain checked only at abstract level and are identified in Book 3 and the evidence record. Actual architect, academic and learner feedback is still pending. The edition must not be represented as independently peer-reviewed.

CHAPTER 1

Architecture as a defensible argument

Learning objectives

Construct a dossier around consequential claims, distinguish constraints from preferences, and keep design views consistent as the architecture changes.

Begin with a decision and its consequences

An architecture assignment becomes manageable when it names a decision. “Design an integration platform” invites unlimited scope. “Choose how three divisions submit service requests while preserving local case ownership and a shared recovery objective” identifies actors, boundaries and consequences. The output is an argument about how a particular arrangement satisfies particular needs.

A claim such as “the service tolerates a thirty-minute vendor outage” needs a mechanism and evidence. The mechanism may be durable local acceptance, bounded storage and a recovery worker. Evidence must examine acceptance during the outage, preservation across restart, and recovery under the vendor's quota. A screenshot of a queue establishes none of those outcomes by itself. Make the missing links visible before choosing products.

This book uses a simple evidence chain: requirement, claim, mechanism, observation, finding and decision. It is an original teaching structure, not a formal proof system. A counterexample can invalidate a claim even if the other links look polished. A successful experiment supports only the conditions it exercised.

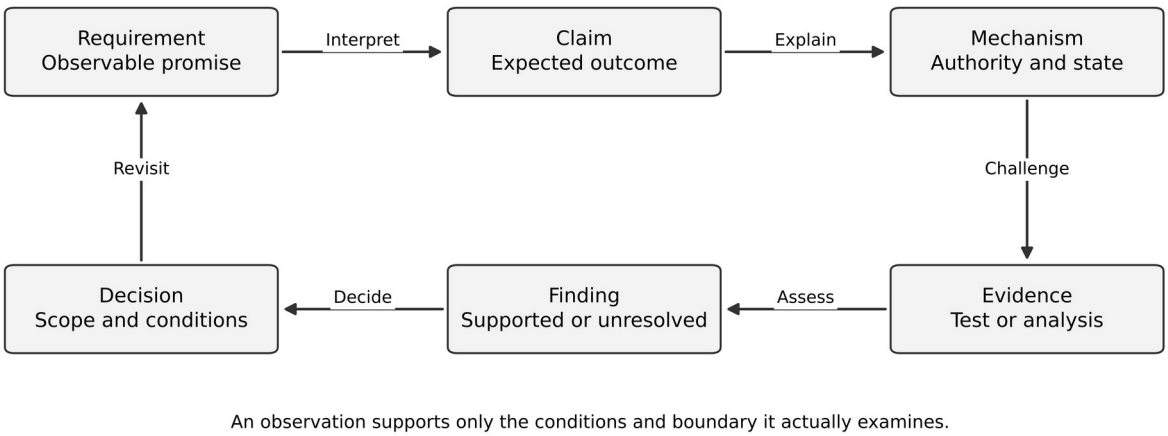


Figure 1 A design decision depends on both its mechanism and the evidence that challenges it

Maintain a small set of connected views

Use a context view to establish scope and ownership; a runtime view for application and data responsibilities; a sequence or state view for important behaviour; a deployment view for placement and

failure domains; and a data view for identity, authority and lifecycle. The C4 model offers an established hierarchy for software structure, while behavioural and data views answer additional questions. [1]

Each view should have a purpose and a version. In an integration design, the runtime view may show an adapter, but the sequence must say what happens when the provider commits and the adapter loses the response. The data model must then contain enough information to recognise that operation. The operations view needs a way to find unresolved outcomes. A diagram that omits the failure is not necessarily wrong; a dossier that nowhere explains it is incomplete.

Use one vocabulary register. If a “request” is a portal receipt and a “case” is a CRM business record, preserve the distinction in schemas, diagrams and user messages. Ambiguous nouns often conceal disagreements about responsibility. Do not solve that disagreement merely by renaming both entities “ticket”.

Constraints and preferences need different treatment

A mandatory condition determines eligibility. A preference helps compare eligible alternatives. If cross-tenant isolation is mandatory, a cheap option that cannot implement it is ineligible; its cost score cannot compensate. Some conditions are negotiable, but the architect must identify who can change them and what consequence follows.

Consider two fictional options. Option A meets a recovery requirement but costs 40 staff-days more to implement. Option B costs less but cannot preserve accepted requests during a provider outage. If preservation is mandatory, the comparison ends until B changes or the business changes its promise. A weighted matrix that ranks B first has hidden the governing decision.

Unknown facts form a third category. A provider may support idempotent creation, but an untested sales statement is not a verified contract. Mark it as an assumption with an owner, a deadline and a consequence if false. This prevents conditional feasibility from turning into an unqualified release recommendation.

Decisions need a history

An architecture decision record should explain context, alternatives, chosen action, consequences and review triggers. Nygard's original ADR description emphasises capturing the forces and consequences around significant decisions. [2] In this course, add evidence identifiers so that a later reviewer can tell why the decision was reasonable at the time.

For example: retain the relational outbox because the first release has one producer and one destination, and measurements show it can meet the recovery objective. Reconsider when a second independent consumer requires retained replay, or when measured polling load becomes material. “Move to Kafka later” is not a review trigger; it states a preferred technology without the condition that would justify it.

An accepted decision is not immutable truth. Preserve the old record, record the new evidence and supersede it explicitly. If a vendor halves a quota, you may keep the architecture and adjust admission policy, or change the integration contract. A revision should explain which causal assumption changed.

Worked dossier fragment

Requirement I-01 states that a valid request receives a durable receipt during CRM failure. Decision D-02 introduces local acceptance and an outgoing intent in one transaction. Data invariant V-01 requires every accepted receipt to reference a handoff intent. Test T-04 kills the process before and after commit. Observation O-04 records whether the request and intent both survive. Finding F-04 closes the local atomicity question while leaving external duplicate handling unresolved.

This fragment is useful because the conclusion is narrower than the whole design. It does not claim that the CRM received the request, that the database survives a regional disaster, or that the user was authorised correctly. Those are separate claims with separate mechanisms. A strong dossier composes them without pretending one test establishes all of them.

Exercises

1.1 Turn “the platform must be reliable” into two distinct, measurable scenarios with different observation points.

1.2 An option scores highest overall but fails a mandatory export requirement. Write a decision paragraph that preserves the business choice without hiding the failure.

1.3 Trace an uncertain payment outcome across five architecture views. Name one contradiction that a cross-view review could detect.

Key points. The dossier should make a decision reviewable through requirements, mechanisms and evidence. Its quality depends on the consistency and limits of the argument, not on the number of diagrams.

CHAPTER 2

Research experiments and architectural evidence

Learning objectives

Design an experiment that can challenge a claim, distinguish models from measurements, and interpret a small sample with explicit uncertainty.

Choose the evidence to fit the question

A proof, a simulation, a benchmark, a case study and a product manual answer different questions. A proof establishes a result under a model; it does not prove that deployed code implements the model. A benchmark measures an implementation under a workload; it does not establish a universal law about a pattern. A manual describes a contract or feature; the chosen configuration must still implement the required behaviour.

Architecture evaluation can reveal risks before full implementation. ATAM explicitly investigates architectural decisions against quality requirements and identifies sensitivity and tradeoff points. [3] The studios use a smaller teaching review; they do not claim to conduct the formal method. The practical objective is to spend investigation effort where a result could change the design.

Write an evidence question with a possible negative result. “Demonstrate our scalable platform” encourages confirmation. “Does the configured worker clear 24,000 pending operations within twenty minutes while receiving twelve new operations per second?” can fail and therefore informs a decision. State what you will change if it fails.

Separate variables and observation boundaries

Independent variables are the conditions you deliberately vary, such as worker concurrency or offered arrival rate. Dependent variables are measured outcomes, such as completion rate or oldest pending age. Controlled conditions include build, schema, hardware class, data distribution and competing traffic. An uncontrolled change can explain an apparent improvement.

For an integration experiment, measure offered operations, accepted receipts, attempted provider calls, distinct remote effects, unresolved operations and completed handoffs. Counting HTTP 200 responses alone can mistake duplicate effects for success. Keep a business identifier separate from an attempt identifier so that retries do not inflate throughput.

An open arrival workload offers requests at the intended rate even when responses slow down. A closed population of clients waits before sending its next request, so offered load may fall when the service is slow. Either model may be appropriate, but they answer different workload questions. Record scheduled, actually sent and completed requests; a saturated load generator can hide the intended demand.

Build a numerical hypothesis before testing

For a constant-rate queue model, backlog changes by arrivals minus successful departures. If arrival rate is a and successful service rate is s, an initial backlog B drains in B/(s-a) seconds only when s is greater than a. This follows from accounting under the model's assumptions; it is not a claim that real traffic has constant rates.

Suppose B is 24,000, a is 12 per second and s is 32 per second. The ideal drain time is 1,200 seconds, exactly twenty minutes. The plan has no allowance for failed attempts, pauses or variable service time. A requirement of “within twenty minutes” therefore needs additional measured capacity, lower admitted demand or a revised recovery promise. The model exposes the risk before a load test.

Little's law relates long-run average work in progress, arrival rate and time in a stable system. [4] It is useful for checking units and rough consistency. Do not substitute it for a peak concurrency limit or assume a growing queue is in steady state.

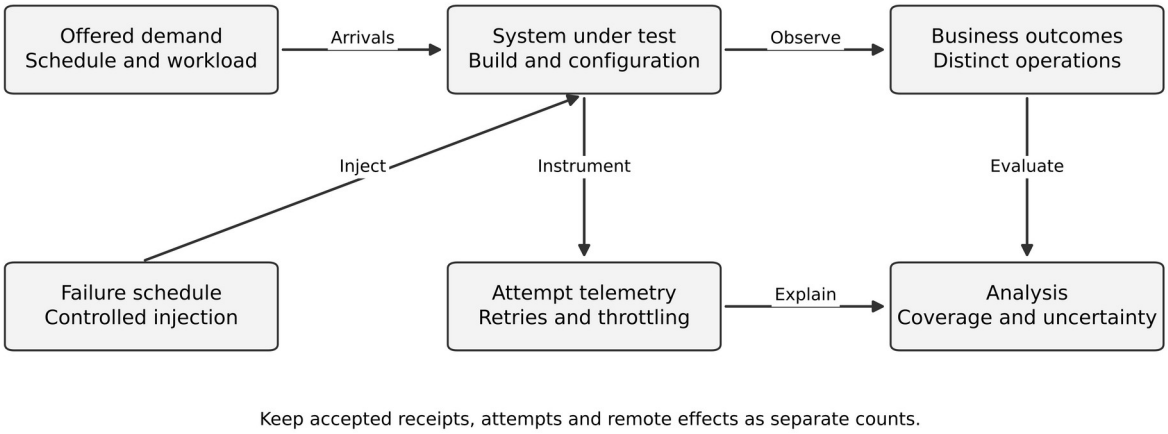


Figure 2 An experiment records demand and business outcomes separately from attempt throughput

Interpret zero failures carefully

Suppose n independent trials each have the same unknown failure probability p. The probability of seeing zero failures is (1-p)^n. Setting this probability to 0.05 gives a one-sided 95 percent upper confidence bound after zero observed failures: p_upper = 1 - 0.05^(1/n). This is a direct binomial-model calculation, not a claim that production failures are independent.

With 1,000 zero-failure trials, the bound is approximately 0.002991, or 0.2991 percent. A flawless sample of that size does not establish a failure probability below 0.1 percent under this method. At least 2,995 zero-failure independent trials are needed for the bound to fall below 0.1 percent. If failures cluster by deployment, host or tenant, treating each request as independent can substantially overstate the information in the sample.

This calculation should not become a release ritual. A thousand requests cannot exercise a disaster that the test never introduces. Statistical uncertainty and scenario coverage are different. A targeted crash at a dangerous transaction boundary may expose a fundamental defect with one trial; a long successful run may miss it entirely.

Read a paper for transfer rather than decoration

For each research critique, identify the source's question, method, assumptions, supported conclusion and limits. Then state your own transfer argument: which property in your design corresponds to the source's model, and which does not? Keep the theoretical statement brief and use your own case to demonstrate understanding.

For example, Parnas's information-hiding argument helps evaluate boundaries around change-prone design decisions. [5] It does not establish that every hidden decision should become a network service. Test your proposed boundary using two actual change scenarios. A separate process may introduce operational coupling that the modularity argument alone does not assess.

Exercises

2.1 Recalculate recovery for 24,000 pending operations, 12 arrivals per second and 28 successful completions per second. Does it meet twenty minutes?

2.2 A test reports zero failures in 1,000 trials and claims 99.99 percent reliability. Explain two distinct defects in the conclusion.

2.3 Design a comparison of two worker policies. Specify variables, business observations, failure injection and a decision rule before generating results.

Key points. Evidence is useful when its method fits the claim and its limits remain visible. Models guide experiments; observations challenge designs; neither should be presented as universal proof.

CHAPTER 3

Integration studio brief and boundary design

Learning objectives

Translate a multi-division integration problem into explicit contracts, quality scenarios and alternative ownership models.

The expanded Meridian assignment

Meridian now operates three service divisions following an acquisition. Each division keeps its existing CRM for at least eighteen months. A shared customer portal must accept service requests and show progress, while each CRM remains authoritative for its own business cases. ERP remains authoritative for assets and entitlement. Twelve engineers are available, including two who support the integration service. These are new assignment assumptions.

The specified peak is sixty valid submissions per second for ten minutes. Normal arrival rate during recovery is twelve per second across the whole portal. One CRM can be unavailable for forty minutes. The portal must preserve accepted receipts during that outage, isolate tenants and stop a noisy division from exhausting every worker. The sponsor wants 95 percent of eligible receipt responses below one second, excluding uploads, and a separate published handoff-recovery objective.

The initial interface investigation finds three different contracts. CRM A enforces a unique external reference and returns the existing case on repeated creation. CRM B offers a search field that is not unique and whose search index may lag. CRM C supplies a durable operation token and a status endpoint, but its documented token-retention window is twenty-four hours. These are fictional provider contracts; do not infer them for real products.

Discover which facts can be promised

The common portal can promise durable acceptance if it controls the local transaction. It cannot promise the same automatic duplicate-prevention guarantee for all three CRMs. A common API should expose meaningful common states while preserving provider-specific uncertainty behind those states. Hiding uncertainty under a universal “complete” response changes the business promise without changing reality.

Define the handoff states as pending, attempting, confirmed, unresolved and rejected. “Unresolved” means the system cannot yet establish whether the remote effect happened. “Rejected” requires a definitive business or protocol outcome. The customer-facing representation may simplify this vocabulary, but the internal model must retain the difference for recovery.

A support case can move from under review to approved independently of handoff attempts. Keep business state and delivery state separate. If a delivery retry overwrites the CRM's later business state, the integration has become a competing authority. The local status projection should preserve source identity and version where the source provides one.

Partition work by authority and failure

Route each request to one division and one authoritative CRM using a server-controlled rule. Persist the route decision with the accepted command. If customer ownership changes later, an in-flight request should not silently move to another CRM unless an explicit transfer process defines that behaviour.

The figure distinguishes one shared acceptance boundary from three controlled delivery paths. Each path has its own concurrency limit and recovery state. This is logical isolation; the diagram does not claim that three worker boxes occupy separate hosts or fault domains. The deployment view must state whether process, compute or account isolation is required by the threat and failure scenarios.

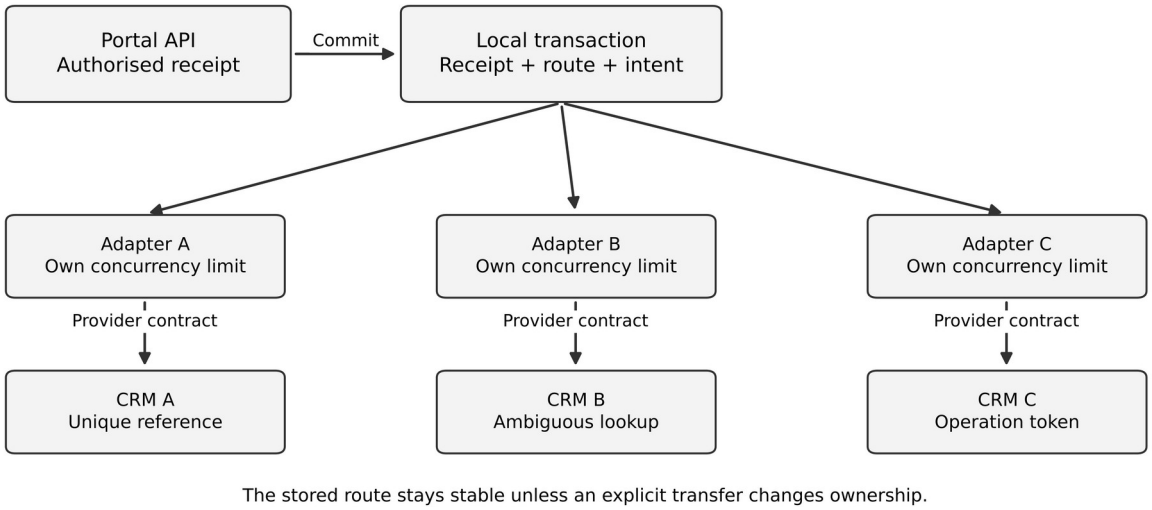


Figure 3 A shared receipt boundary feeds separately controlled CRM delivery paths

A shared relational database can store receipts and outbox intents while logical queues select work by division. This reduces platform count but requires fairness and query design. Separate brokers or databases offer stronger operational separation at additional cost. Choose the boundary that addresses an identified failure, and test whether a slow CRM can still consume shared database connections or storage.

Three credible alternatives

Option A is a portal application with a relational outbox and dedicated adapters. Option B adds a managed broker between publication and adapters, with a transactionally recorded outgoing intent. Option C uses an integration platform with provider connectors and a durable workflow. None removes the need to understand remote creation semantics.

Option	Main strength	Main cost or uncertainty
Relational outbox and adapters	Small platform surface and explicit local atomicity	Polling load, fairness and replay need implementation
Outbox plus broker	Independent consumption and buffering	Another durability boundary and operating contract
Integration platform	Managed workflow and connector operations	Connector semantics, export, pricing and exception control

A broker becomes valuable when consumers need independent progress or retained delivery. It does not make the database and broker commit atomically by existing on the diagram. AWS's outbox guidance describes storing the state change and outgoing intent together and handling possible duplicate delivery. [6] The assignment must then explain the broker publication checkpoint and consumer transaction boundary.

Your first submission

Before reading Chapter 4, produce a problem brief, eight quality scenarios, a fact-ownership table, two alternative runtime views and a contract for one uncertain handoff. Mark the unresolved provider behaviours. Include a proposed recovery objective that you can defend quantitatively; the brief deliberately does not supply a feasible value for every provider.

Give the support team a concrete role. For CRM B, they may need a controlled worklist showing candidate matches and evidence. A manual step is part of the architecture when correctness depends on it. Estimate its volume and deadline; “operations will handle exceptions” is not a bounded design.

Exercises

3.1 Explain why a successful retry contract for CRM A cannot simply be declared for CRM B.

3.2 Define a stable routing rule for a request whose customer changes division after acceptance.

3.3 Compare logical worker isolation with separate deployments. State a failure scenario that justifies the additional deployment boundary.

Key points. A unified experience can coexist with different external guarantees. The architecture must preserve those differences where they affect correctness, recovery and support work.

CHAPTER 4

Integration contracts and competing designs

Learning objectives

Specify a duplicate-safe local contract, analyse external uncertainty, and compare implementation choices without treating product features as end-to-end guarantees.

The local submission contract

The submission endpoint authenticates the caller, establishes authorised tenant scope and validates the request. It accepts a tenant-scoped operation key and binds that key to the authorised owner and a canonical representation of the relevant payload. In one local transaction it stores the receipt, the key's result, the selected division and the outgoing intent. Reusing the same key with different relevant content is a conflict.

The response says that Meridian has accepted responsibility for the handoff. It does not say that a CRM case already exists. A 201 response can represent creation of the receipt resource; a 202 response can represent accepted processing under a different resource contract. HTTP defines these response semantics, while the business guarantee remains the application's responsibility. [7]

A repeated request must still pass current access checks. The operation key is an identifier, not a credential. If ownership has changed, returning a cached body without authorisation can expose another user's data. Define the key retention period and what happens to late retries after expiry. Indefinite retention is not the only possible design, but expiry must not create an invisible duplicate risk.

The worker lease does not control the provider

A worker claims an intent with a lease and attempt identity. It sends the original remote operation identity and records the observed result. If the worker pauses beyond lease expiry, another worker may claim the work. The old worker can still awaken and send a request unless the protocol prevents it. A database lease alone therefore does not prove one external effect.

A fencing token can help when the resource receiving an operation understands and rejects stale tokens. Sending an increasing number to a provider that ignores it provides no protection. For the three fictional CRMs, provider idempotency and outcome lookup remain decisive. Keep the lease's local ownership guarantee separate from the remote contract.

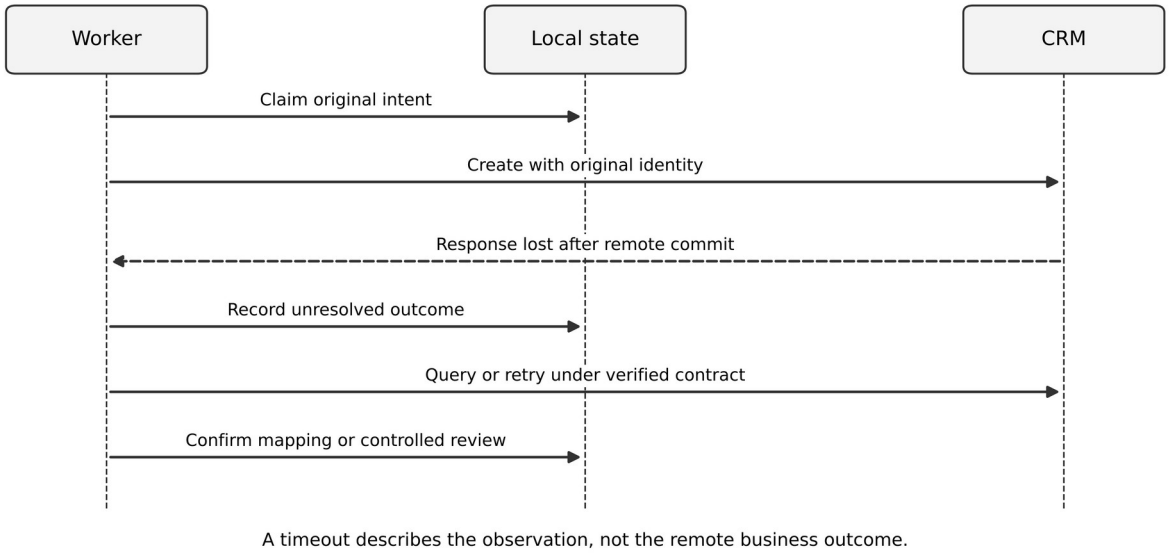


Figure 4 A crash after remote commit leaves an unresolved operation rather than a known rejection

For CRM A, retry using the original unique reference and verify the returned case belongs to the intended operation. For CRM C, persist the operation token and query it; before its retention window ends, resolve or escalate incomplete work. For CRM B, a search returning no match may reflect index lag. Automatic creation with a new identity could duplicate a successful earlier operation. The safe course depends on an agreed waiting, reconciliation and manual-decision policy.

A concrete unresolved-operation record

Retain the receipt ID, tenant, division, provider, original operation ID, payload digest, first-attempt time, last observed category, candidate remote references and resolution history. Store only information necessary for recovery and the approved audit policy. Sensitive request content belongs in controlled storage rather than broad logs.

Resolution itself needs concurrency control. If a support agent confirms a candidate while an automated retry is still active, the two actions can race. Move the item into a controlled resolution state, prevent new attempts under the local policy, and record the final mapping atomically. A late response from an earlier attempt must be reconciled against that mapping rather than overwriting it.

Compare the two technical designs

In the direct outbox design, the adapter reads pending intents from the database. Index by delivery state, next eligible time and division, and bound each polling batch. Record successful handoff state without deleting all evidence immediately. The database carries both transactional load and work-selection load; test this mixture at the case peak.

In the broker design, a publisher sends an outbox intent and records publication progress. A crash after broker acceptance but before the checkpoint may publish twice. The adapter must therefore deduplicate its local workflow state and still use the remote operation contract. Broker acknowledgment means what the broker contract says; it does not by itself establish CRM completion. RabbitMQ distinguishes publisher confirmations from consumer acknowledgments, illustrating this separation. [8]

A workflow platform can implement orchestration, timers and retry history, but inspect its connector's exact treatment of timeouts. Does it reuse the same provider operation identity? Can operators stop a pending retry? Can the workflow history be exported and reconciled? A feature named “retry” is neither sufficient nor necessarily safe.

Azure and AWS mapping as an investigation

One implementation could evaluate Azure Container Apps, Azure Database for PostgreSQL and Service Bus. Another could evaluate ECS with Fargate, RDS for PostgreSQL and SQS. These are role mappings from the vendor documentation categories, not interchangeable deployment recipes. [9] Keep the architecture contract stable while investigating transaction, ordering, retention, identity and recovery differences.

The first decision can remain provisional: choose direct polling for the pilot because there is one handoff path per receipt and the team can operate the database, then introduce a broker if measured fairness or independent consumption needs justify it. The pilot is conditional on provider semantics and peak-load evidence. A different team with an established integration platform may rationally choose another option.

Work the unknown outcome through two architectures

Take a single accepted Meridian request, R17. The portal commits its receipt and delivery intent locally. A worker sends a create-case command to CRM B. CRM B creates case C91, but its response is lost. The worker records a timeout. A subsequent CRM search returns no result because the searchable view is behind. The evidence is compatible with both a successful create not yet visible and an unsuccessful create. Retrying blindly may create a second case; treating the timeout as failure may cause support staff to act on a false status.

Alternative A uses a durable local outbox and an uncertainty state. The receipt transaction commits R17 and its outbox entry together. Delivery may happen more than once, as the transactional-outbox guidance warns. [6] The worker stores an attempt identifier before sending, records the timeout, and moves the command into a reconciliation state. A retry is automatic only when the destination contract makes it safe. For CRM B, the negative lagging search is not sufficient evidence. A reconciliation operator may need to inspect the destination with stronger evidence before authorising another create.

Alternative B changes the accepted service boundary. The portal accepts a durable request for staff-assisted handoff and promises a tracked outcome, while reserving automatic case creation for CRMs with an adequate duplicate-control contract. The queue item becomes a work assignment whose completion requires a verified destination case ID. This reduces some automation benefits and adds staffing cost, but it can be a more honest first release than advertising automatic exactly-once handoff to an incompatible destination. It does not remove duplicate risk if staff also retry without checking; the same evidence rule must govern the manual path.

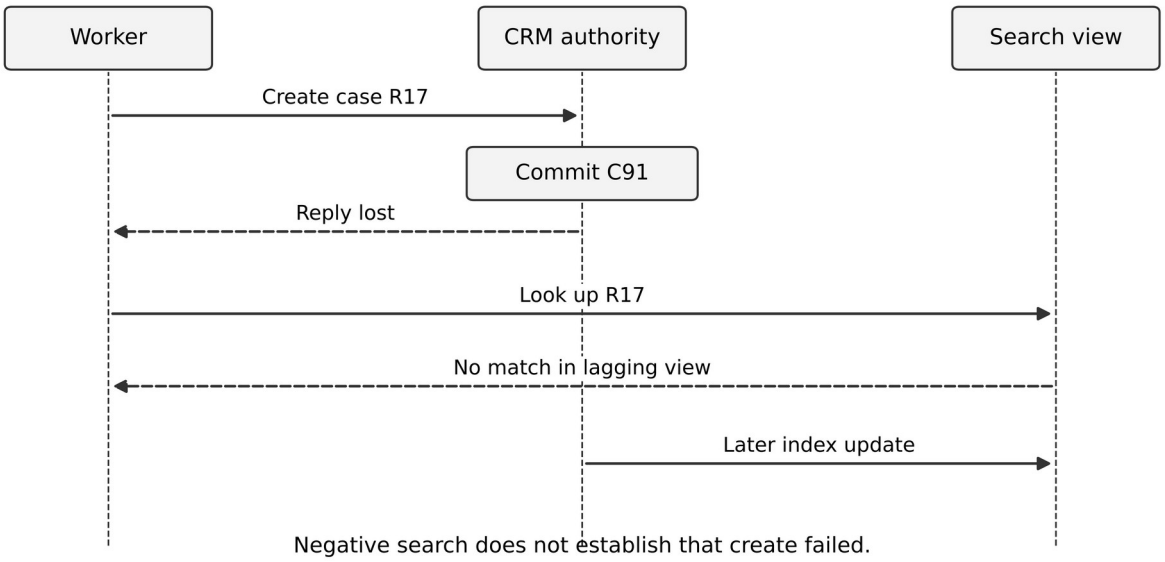


Figure 5 A lost response and lagging lookup leave the destination outcome unknown

The figure is an original failure trace. The destination commit lies before the lost reply. The dashed lookup result is an observation of a delayed view, not evidence that the write was rolled back. Read the trace from top to bottom. The durable attempt record and the case identifier discovered later connect the evidence across systems. The trace should appear in the design review because a success-only sequence hides the decisive risk.

For CRM A, uniqueness of an external reference permits a different recovery argument: submit R17 under that reference and resolve duplicate responses to the existing case, provided the uniqueness constraint covers the relevant scope and persists through the intended retry horizon. For CRM C, the twenty-four-hour idempotency window supports retries only within its declared contract. A command held for longer must not be replayed with a new key merely to avoid an error. Confirm the prior outcome or route to controlled reconciliation.

The outbox owns evidence of what the portal intends to deliver. It is not a second authority for the CRM's case status. When status comes back, retain the CRM identity and version information needed to interpret it. A delayed status message can be older than the last observed status. Comparing arbitrary wall-clock timestamps across systems is not sufficient to settle business order unless the contract provides the necessary semantics.

An architecture decision record should therefore contain a release condition: demonstrate destination duplicate behaviour, lookup visibility and retention in an authorised test environment. Record what the local experiment cannot establish. The supplied deterministic recovery model validates the course state transitions under declared contracts; it does not verify any real CRM. The production acceptance evidence must come from that destination's actual behaviour and agreement.

Exercises

4.1 A worker's lease expires before its remote request completes. Explain why a second worker can create a duplicate and identify the contract that prevents it for CRM A.

4.2 Design the resolution transition when an operator finds two possible CRM B matches. What must the system avoid claiming?

4.3 Write an ADR comparing polling and a broker for this case. Include a measurable trigger for revisiting the choice.

4.4 CRM B times out after create and its lagging search returns no match. Give two executions consistent with this evidence and defend the next action.

Key points. Local transactions, publication acknowledgments, worker ownership and external business effects are separate guarantees. A defensible integration design connects them without silently strengthening any one of them.

CHAPTER 5

Integration evaluation and a changed quota

Learning objectives

Estimate backlog and recovery, evaluate fairness under a shared limit, and revise the integration design when a provider changes its contract.

Establish the recovery scenario

The sponsor now proposes that one CRM's forty-minute outage must be followed by recovery within thirty minutes. For the experiment, that CRM receives a constant six accepted requests per second throughout the outage and recovery. Its permitted successful handoff rate after recovery is twenty per second. These simplifying assumptions are local to this calculation.

The outage creates 6 × 40 × 60 = 14,400 pending handoffs. Net drain is 20 - 6 = 14 per second, giving about 1,028.57 seconds, or seventeen minutes nine seconds. This is feasible in the ideal model, leaving almost thirteen minutes within the proposed thirty-minute window. The model excludes retries, competing clients, variable call duration and a provider that recovers gradually.

Now distinguish a successful handoff quota from an attempt quota. If the vendor limits calls to twenty per second and ten percent of attempts fail without completion, effective throughput is approximately eighteen per second only under the stated simple success model. Net drain becomes twelve per second, and recovery takes twenty minutes. Correlated failures or extra lookup calls can reduce it further.

Observe the right quantities

Record accepted receipts and distinct completed handoffs by division. Also record attempts, lookup calls, throttled calls, unresolved outcomes, oldest pending age and database wait time. An aggregate completion graph can look healthy while one division starves. Present both aggregate and per-division observations.

The experiment should interrupt the CRM at the declared peak, restart workers during recovery and introduce duplicate deliveries. Verify that every accepted receipt ends in a known handoff state or a visible unresolved work item. A count of “processed messages” is insufficient if the same receipt was counted repeatedly.

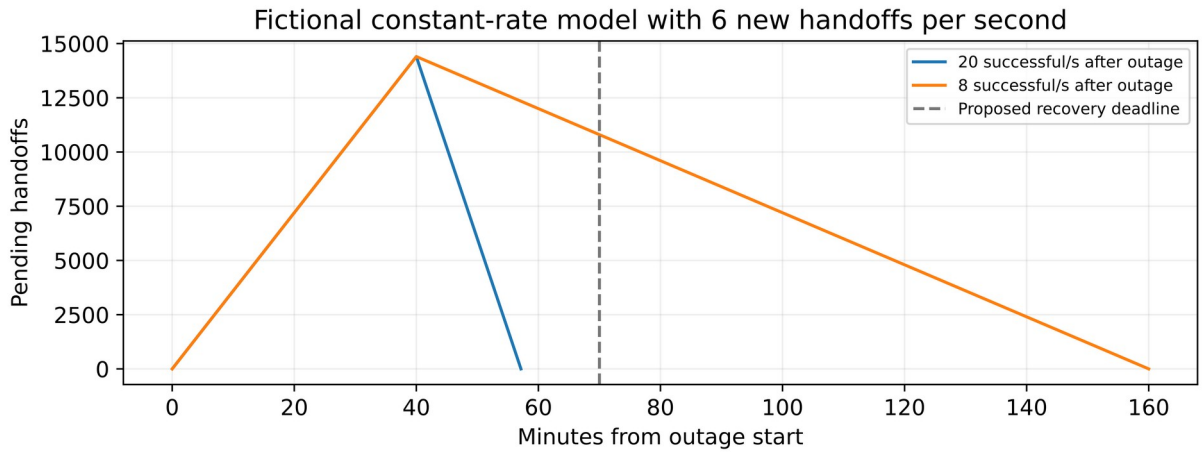


Figure 6 Backlog growth and recovery depend on successful service after shared limits and new arrivals

The change card

After the first design review, the vendor announces a shared account quota of eight calls per second for the affected CRM. All tenants in that division share the quota, and status lookups consume it too. The observed incoming demand remains six new requests per second. The old scaling rule adds workers whenever backlog grows.

At eight successful creations per second and no other calls, net drain is only two per second. The same 14,400-item backlog needs 7,200 seconds, or two hours. Even this optimistic bound fails thirty minutes. Adding workers cannot remove the shared quota; it may increase throttling and consume capacity on retries.

To drain in thirty minutes under the constant-rate model, successful creation capacity must be at least 6 + 14,400/1,800 = 14 per second. Because lookup and failed calls also consume quota, the required account call quota would be higher than fourteen unless those other calls are eliminated or separately budgeted. The calculation identifies a negotiation and contract issue, not merely a compute-sizing issue.

Revise the recommendation explicitly

Possible responses include negotiating capacity, reducing admitted work with a clear customer promise, changing the recovery objective, prioritising critical cases, or moving selected work to a provider-supported bulk mechanism. Each changes a business or technical contract. Silent dropping of accepted receipts violates the original promise.

If prioritisation is chosen, define classes, allocation and starvation limits. A strict priority rule can indefinitely delay ordinary requests during sustained critical demand. A weighted schedule may reserve a minimum share while borrowing unused capacity. The support team needs the age distribution and an escalation rule for work approaching its deadline.

Use a shared rate-control mechanism whose scope matches the vendor account. A limiter in every replica can multiply the aggregate rate. Test coordinated operation, process restart and temporary loss of limiter state. The detailed implementation can vary; the invariant is that the combined offered load respects the agreed quota within its specified measurement window.

Assess the economic and operational alternatives

Suppose a fictional quota upgrade costs 900 euros per month, while manual resolution consumes 30 staff-hours per week at an internal planning value of 45 euros per hour. The annual effort is 30 × 52 = 1,560 hours, valued at 70,200 euros. The upgrade is 10,800 euros annually. This does not prove the upgrade is sufficient or economically preferable: establish how many manual hours it actually removes and whether the new quota meets the requirement.

The deliberately explicit units prevent a common error: multiplying weekly hours by months or confusing capacity released with cash saved. Internal time value is not necessarily a budget reduction. Present the causal benefit and its uncertainty alongside the expenditure.

Studio assessment

Submit a revised dossier with the original and changed assumptions, two recovery calculations, a failure experiment and an ADR explaining the quota response. Include a research critique on the outbox or end-to-end correctness boundary. Explain which guarantees remain conditional on the provider and which are controlled locally.

Use the common rubric in Chapter 16. An answer that changes the thirty-minute objective with explicit sponsor agreement can be better than one that preserves the number by inventing unlimited vendor capacity. The architect's task is to make the consequence visible and propose feasible choices.

Exercises

5.1 Calculate the minimum successful handoff rate needed to clear 14,400 items in thirty minutes while six new items arrive each second.

5.2 A dashboard reports 100 completed attempts per second across all divisions. Give three reasons this may not establish successful recovery.

5.3 Propose a response to the eight-call shared quota that preserves accepted receipts. Explain its effect on customer expectations and support work.

Key points. Recovery depends on net successful capacity, not worker count. A changed external limit can require a changed business promise, and the revised decision must show that dependency.

CHAPTER 6

Transaction studio brief and business invariants

Learning objectives

Define transaction correctness in business terms, separate admission from confirmation, and identify where a high-volume workload actually creates contention.

Aster launches a limited stock event

Aster's second-stage assignment is a limited-stock sales event with a peak of 500 checkout attempts per second for ten minutes. A single promoted product may receive 70 percent of attempts. There are 2,000 units allocated to the online event, and the business forbids confirmed sales beyond that allocation. The two warehouses retain their own physical stock systems; the event allocation is a separately agreed selling allowance.

The payment provider supports an operation identifier and an outcome lookup under a documented twenty-four-hour idempotency window. Authorisation may take up to thirty seconds in the normal design scenario. A timeout leaves the outcome unknown. A reservation normally expires after five minutes, but the exact policy for an unresolved payment must be designed. These are fictional contractual assumptions for the assignment.

The initial team proposes separate catalogue, cart, order, inventory, payment and notification services. The product manager asks for a simple customer message: “Your order is confirmed.” Before accepting the service diagram, define what must be true for that message to be honest.

Derive the invariants

Let A be the remaining event allocation, R active reserved units, C committed units and L the initial allocation of 2,000. If no replenishment or adjustment occurs, a useful accounting invariant is A + R + C = L, with each quantity non-negative. A reservation moves quantity from A to R; confirmation moves it from R to C; release returns it from R to A. A duplicate transition must not move quantity twice.

This model describes the selling allowance, not the physical warehouse stock. A separate reconciliation process must explain cancellations, damage, replenishment and fulfilment adjustments if they enter scope. Do not claim that a correct event ledger proves the warehouse has the goods unless the allocation agreement makes that connection.

An order is confirmed only when the relevant reservation is committed and the required payment authorisation is verified. The policy may allow a different order contract, such as backordering, but that requires a different message and state model. An HTTP success for checkout receipt is not automatically an order confirmation.

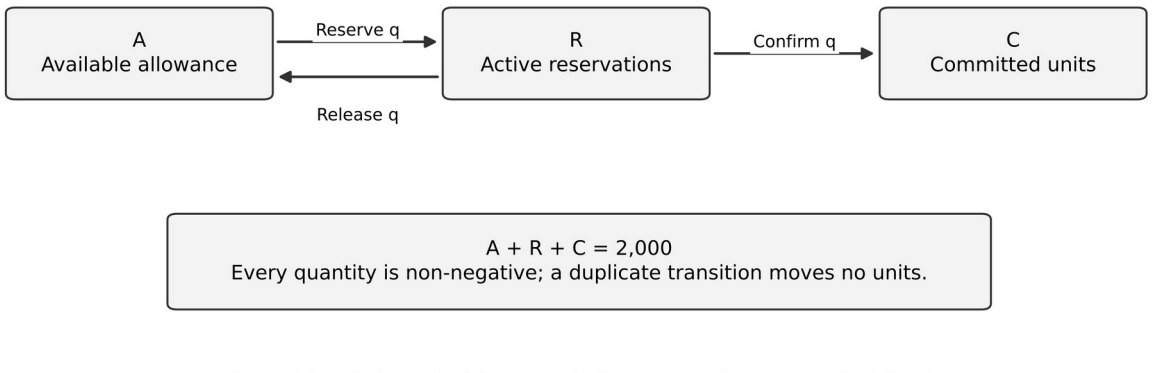


Figure 7 Reservation transitions preserve a fixed selling allocation under the stated event model

Find the contention boundary

Five hundred requests per second does not mean five hundred independent database keys. Seventy percent directed at one product gives 350 attempts per second at one allocation boundary. Partitioning unrelated products across servers may leave that hot product unchanged. Measure contention and admission behaviour at the actual invariant.

A local atomic conditional update can reserve quantity when enough remains, together with the reservation record. Under a suitable database contract, competing attempts either reserve safely or observe failure and respond accordingly. The design must define transaction retries, operation identity and how the application distinguishes out-of-stock from transient contention. PostgreSQL's isolation documentation illustrates why application behaviour must account for the selected level and possible transaction aborts. [10]

A read cache can serve a display of approximate availability, but it should not independently authorise scarce-stock consumption unless the architecture assigns it that responsibility with a correct protocol. The storefront may show “limited stock” while final reservation determines the outcome. Customer communication is part of consistency design.

Two first designs

Design A keeps order and event allocation in one modular application and one transactional database boundary. The payment call remains external and is coordinated through durable state. This reduces distributed transaction complexity for the critical local invariant. It can still scale stateless request handling and separate noncritical catalogue reads.

Design B gives inventory and orders independent services and stores. An inventory reservation becomes an explicit contract with a reservation ID, quantity, owner, expiry, version and terminal outcome. The order coordinator must handle partial completion and compensation. Independent ownership or a shared multi-channel inventory capability can justify this design, but merely drawing more boxes does not establish that benefit.

Both designs need provider idempotency and reconciliation for payment. Both need bounded admission when the event is sold out or a dependency is degraded. Both need a way to explain each customer's outcome after a process crash. The choice concerns the placement of local authority and its operating cost, not whether one pattern is inherently modern.

Your first submission

Write five invariants, a state model, two alternative runtime views and a sequence for payment authorisation arriving after reservation expiry. Define the exact customer message at each boundary. Include a capacity hypothesis for the hot product and an experiment that can reveal overselling even when response latency is good.

Keep card handling outside the teaching implementation by using fictional provider tokens. The exercise is about architectural responsibility and uncertainty, not implementing payment credentials or claiming payment-industry compliance. A real system must separately satisfy its actual contractual and security obligations.

Exercises

6.1 Starting from A = 2,000, R = 0 and C = 0, reserve three units, confirm two of them under a policy that permits partial confirmation, and release one. Show the final values and invariant.

6.2 Explain why partitioning the catalogue into ten equal groups may not solve the promoted product's contention.

6.3 Define an order-confirmed event contract that distinguishes verified business completion from acceptance of a checkout request.

Key points. Transaction architecture starts with business invariants and authority. Workload concentration, external payment uncertainty and the meaning of confirmation matter more than the number of services.

CHAPTER 7

Reservations payments and coordination

Learning objectives

Compare local and distributed reservation protocols, design explicit compensation, and prevent late messages from reviving an invalid order.

Model the order as durable progress

The order coordinator records checkout received, reservation active, payment pending, payment unknown, confirmed, cancellation pending and resolved cancellation as appropriate. These are logical business and delivery states; implementation may represent them in several related records. Every transition names its triggering evidence, guard, durable effect and operation identity.

A timeout while authorising payment moves the workflow into uncertainty. It does not justify recording “declined”. The coordinator retains the provider operation identity and queries or retries according to the verified contract. If the provider confirms authorisation, the order still must verify that a usable reservation exists. Payment success alone does not restore stock that was already released.

The original Sagas work describes long-lived transactions organised into smaller transactions with compensation. [11] In this assignment, compensation is an explicit business action such as releasing a reservation or requesting a payment void. It can fail or remain pending; it is not a magical reversal of history.

Make expiry and confirmation compete atomically

In Design A, expiry and confirmation can update the reservation and order within the local transaction boundary. Both require the expected active reservation version. If expiry commits first, confirmation cannot consume that expired reservation. If confirmation commits first, expiry must see that it is no longer releasable. The terminal state establishes which effect is allowed.

In Design B, the inventory service owns that arbitration. The order service calls a confirm-reservation operation with the original reservation ID and expected contract. A response timeout can leave confirmation uncertain, so the inventory service must offer a duplicate-safe retry or a query of the same reservation. Splitting the service adds another uncertain-outcome boundary even before considering payment.

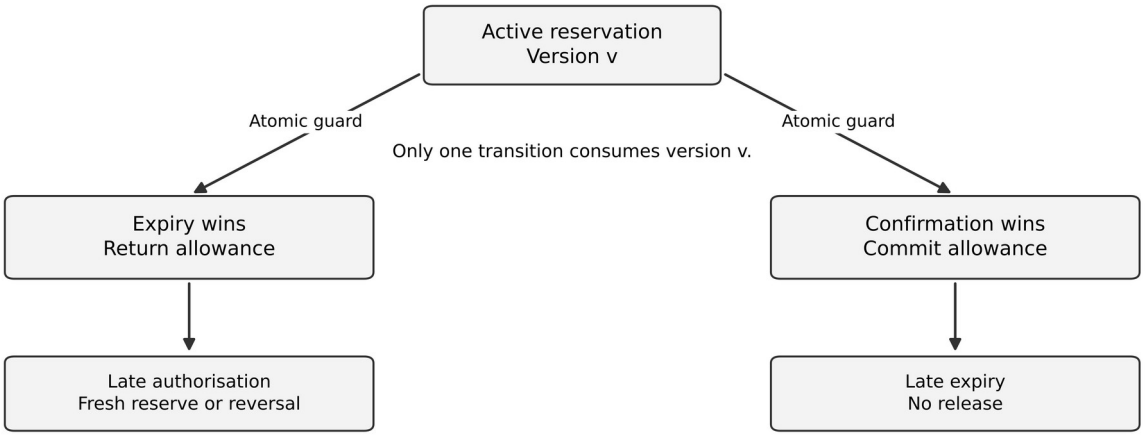


Figure 8 Expiry and confirmation contend for one authoritative reservation transition

Do not solve the race using timestamps alone. Two processes can read an apparently unexpired record and act concurrently. A timestamp is useful input to the guard; the authority must apply the guard and state change atomically. Specify whose time determines expiry and how delayed messages are interpreted.

The late authorisation case

Suppose the reservation expires at 12:05 and is released. At 12:06, the provider confirms a payment authorisation that actually succeeded earlier. One defensible policy attempts a new reservation under a new reservation identity while retaining the same payment identity. If fresh stock is available and policy permits, the order can proceed. If it is not, the coordinator requests a void or another provider-supported reversal and keeps the order in cancellation pending until that outcome is known.

The system must not silently charge again, invent stock or mark cancellation complete merely because a void request was sent. Record the compensation identity, its result and the person or automated process responsible for unresolved work. A customer message can explain that the order could not be confirmed and the payment authorisation is being resolved, using wording approved for the actual provider behaviour.

A stricter alternative holds the reservation while payment is unresolved, up to a bounded deadline. This reduces one race but can lock scarce stock behind slow or malicious attempts. Evaluate maximum holds, rate limits, abandonment and fairness. The policy is a tradeoff between selling availability and the burden of late-result handling.

Can allocation partitions avoid central contention

An alternative gives each independent seller a bounded sub-allocation. If two partitions each receive 1,000 units and cannot sell beyond their local allowance, the combined allowance remains 2,000. This can reduce coordination for local sales under the simplified model. It also permits one partition to sell out while another has unused units.

Transferring allowance requires its own safe protocol. If a transfer is credited at the destination before it is durably removed or fenced at the source, both sides may spend the same units. Model transfer as an accountable operation with pending state and recovery. Do not call this a solution to every multi-region inventory problem; it is a deliberate restriction of what each partition may promise.

Choreography and orchestration

A central coordinator makes the order's progress and exceptions explicit but becomes a responsibility to operate and evolve. Event choreography distributes reactions among participants but can make the full lifecycle harder to reconstruct. Either can preserve the invariants if contracts and ownership are clear.

For this teaching case, a durable order coordinator is a defensible starting point because payment, reservation expiry and compensation require a visible process. Notification can remain an independent consumer because an email failure need not reverse a valid order. Separating the critical confirmation path from ancillary effects reduces unnecessary failure propagation.

Compare a single stock authority with allocated selling rights

The studio's 2,000-unit allowance is deliberately finite. Begin with the invariant already defined: available rights, active reservations and committed sales sum to 2,000, excluding separately governed replenishment or adjustment. The architectural question is where a command can atomically prove that consuming a right preserves this invariant. The following alternatives are original designs for analysis, not ready-made production protocols.

Alternative A keeps one logical stock authority. All commands for the hot SKU reach a service that atomically checks availability, decrements it and records a reservation identity. Its durable state may be replicated, but clients do not independently update arbitrary replicas. A repeated reservation command resolves to the same reservation. Confirmation changes a reservation state once; expiry competes through the same guarded state transition. This centralises the invariant and simplifies the proof. It also places hot-SKU throughput and progress during some failures behind that authority.

For the 350 hot-SKU attempts per second, the offered workload is not the number of successful sales. Once the 2,000 rights are consumed, further attempts should receive a defined out-of-stock or waiting response. With one unit requested per successful attempt, no replenishment and every attempted reservation succeeding initially, 2,000 divided by 350 is about 5.71 seconds. This is a deliberately extreme saturation case. It explains why a benchmark reporting only total request throughput can conceal the tiny period in which concurrent reservation correctness matters.

Alternative B allocates rights to regions. Before a failure, give North 1,200 rights and South 800. Each region can reserve only its own unspent allowance. Local transactions preserve local conservation; the sum preserves the global ceiling if allocation and transfer cannot duplicate a right. A disconnected North that has consumed all 1,200 cannot borrow South's unspent rights by guessing. North must wait, reject or accept a clearly tentative request. The benefit is local progress while rights remain; the cost is stranded stock, allocation policy and a more complicated recovery argument.

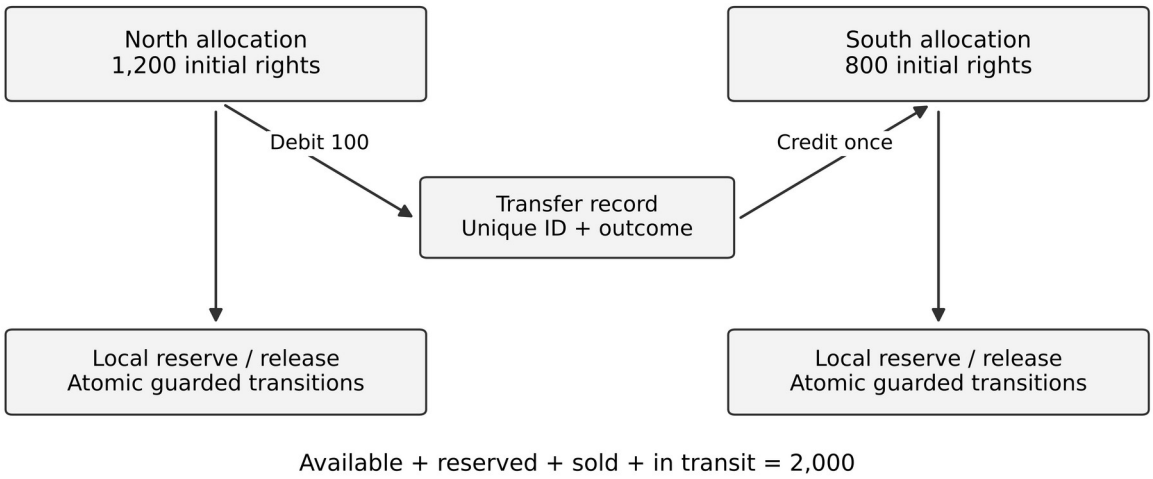


Figure 9 Regional progress requires conserved rights and a separately controlled transfer

A rights transfer is a stateful operation, not a pair of unrelated counter updates. One possible design removes 100 available rights from North into a durable in-transit record, then credits South under a unique transfer ID. A retry cannot credit South twice. North cannot reissue the rights merely because its acknowledgement is missing. Conservation now includes available rights, reservations, sales and rights in transit. Recovery must resolve the transfer before either side can spend those rights again. This sketch exposes obligations; it is not a proof of a complete distributed transfer protocol.

Consider a partition immediately after North records the transfer but before South acknowledges. In this design the 100 rights may be temporarily unavailable. If an operator restores North from an old snapshot and simply starts spending the old counter, the global bound can fail. Recovery therefore requires fencing obsolete writers and restoring allocation history consistently. An allocation method that is safe during normal operation but unsafe during disaster recovery does not meet the invariant.

Design dimension	Single logical authority	Allocated regional rights
Decision boundary	One guarded stock command	One guarded local spend within an allocation
Partition behaviour	Some callers cannot reserve	A region can reserve until its rights run out
Scarcity handling	All unspent stock is visible to the authority	Stock can be stranded in another region
Recovery obligation	Restore one authoritative history and fence stale writers	Also reconcile allocations and in-transit transfers
Evidence needed	Concurrent command histories and failover tests	Those checks plus conservation across transfer and recovery

Aster's choice depends on measured latency, partition requirements, demand skew and the team's ability to operate the extra protocol. A two-region marketing requirement alone does not justify the second design. For a first implementation, one stock authority may be preferable if its measured behaviour meets the service contract. If regional autonomy during disconnection is essential, explicit selling rights provide a design to evaluate, with reduced utilisation accepted as part of the tradeoff.

Payment timing does not collapse into the stock transaction

Compare reserving before payment with authorising payment before reservation. Reserving first protects availability for the customer during payment, but abandoned checkouts consume temporary stock and can be abused. Authorising first avoids holding scarce stock for users who cannot pay, but can leave an authorised payment without stock, requiring a void or other approved outcome. Neither ordering eliminates partial failure. Decide which intermediate state the business is willing to own.

For this studio, maintain a reservation before confirming an order and verify the provider's payment outcome. If payment succeeds after the reservation expires, do not move an expired reservation to confirmed merely because a callback arrived. Attempt a new reservation under a defined policy or initiate the permitted payment recovery. A void and a refund may have different provider semantics and business consequences; the case must state which operation is available at that payment state.

The original Sagas paper treats compensation as a semantic correction and permits other work to observe intermediate effects. [11] For Aster, a compensating payment command can itself time out. Keep a durable compensation-pending state with an owner and reconciliation procedure. A customer-facing message should distinguish confirmed order, payment pending and recovery pending. Hiding all of them behind failed invites customers to retry an action that may already have taken effect.

Separate two proofs. The stock proof shows that guarded transitions cannot spend more rights than exist. The workflow argument shows how each admitted order reaches an acceptable completed or recovered state under declared provider and recovery assumptions. The second is not established by the first. A fault-injection test should therefore inspect both the stock invariant and the eventual business outcome, including cases where the latter remains unresolved and needs intervention.

Exercises

7.1 Trace both interleavings of reservation expiry and confirmation. State which transition must fail in each ordering.

7.2 Evaluate holding a reservation until payment is known versus releasing at a fixed deadline. Identify a business risk and an operational control for each.

7.3 A partition transfers 100 units of allowance to another region. Design the invariant and unresolved state needed to avoid double spending of that allowance.

7.4 North has 1,200 rights and South 800. North starts transferring 100, then loses the reply. Define a conservation equation and explain why North cannot simply reclaim the 100.

7.5 Compare payment-first and reservation-first for scarce stock. Include a late payment callback and a timed-out compensation in both analyses.

Key points. A state transition is safe only at the authority that owns its invariant. Late results and failed compensation must remain visible, and every new remote boundary introduces another outcome to reconcile.

CHAPTER 8

Transaction experiments and failure review

Learning objectives

Test invariants under adverse event orderings, evaluate admission control, and interpret latency distributions without concealing failed requests.

Test the boundary before the throughput

Begin with a tiny allocation and a known sequence. With one unit available, issue two concurrent reservations and require exactly one committed reservation. Retry the winner with the same operation ID and require the same result without another decrement. Release it twice and require the second release to have no additional effect. These tests expose correctness before load complicates diagnosis.

Now inject a crash after the provider authorises payment but before local recording, a duplicate callback, expiry before the callback, and failure during compensation. For each schedule, state the expected invariant and customer-visible status. A successful test is not merely “no exception thrown”; it accounts for allocation, reservation, order and payment state.

The supplied local state experiment enumerates several event schedules under a small deterministic model. It does not reproduce a database isolation implementation, distributed clocks or provider behaviour. Its purpose is to expose missing transition rules. Follow it with an integration test in the actual chosen environment when preparing a real release.

Admission is a business control

When demand exceeds useful capacity, unbounded pending work can worsen the customer outcome. Aster can reject new checkouts with an honest retry or sold-out response, accept them into a bounded pending state, or use a waiting room whose admission rule is explicit. The correct choice depends on the promise made before payment and reservation.

Do not count a fast rejection as a successful sale. Define separate indicators for storefront availability, valid checkout admission, confirmed orders, unresolved payments and time to final outcome. A design can meet a response-latency target while most customers fail to buy. The service objectives should reflect the actual journey.

A synthetic experiment report

Consider the following invented test results for the same five-minute offered workload of 500 attempts per second, or 150,000 attempts. These are teaching data, not observed benchmark results.

Policy	Admitted attempts	Confirmed orders	Unresolved payments at cutoff	p95 receipt latency
Unbounded admission	150,000	1,990	600	4,800 ms
Bounded admission	12,000	1,990	12	620 ms
Strict sellable admission	2,300	1,985	4	410 ms

The table alone cannot select a winner. Define whether admitted attempts include duplicates, how many were valid distinct customers, what happened to rejected callers, and why confirmed orders remain below 2,000. The unresolved count is measured at a cutoff, so later completion and customer support effects matter. The final policy may sacrifice a few potential confirmations while improving certainty, but that is a business tradeoff requiring evidence.

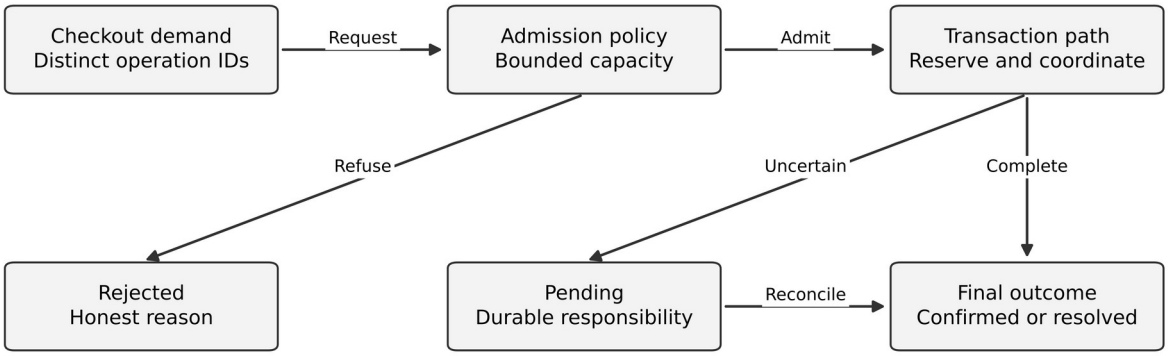


Figure 10 Admission limits protect the critical transaction path while preserving explicit rejected and pending outcomes

The next experiment should investigate the reason for the five-order difference, not simply repeat the same benchmark with more hardware. It may reflect a timeout budget, an inventory hold rule, a provider limit or an observation cutoff. Measure event histories for the affected orders and distinguish unfinished work from permanent loss.

Tail latency and correlated dependencies

Dean and Barroso explain why occasional slow components become consequential when a request depends on many components. [12] Aster should therefore keep optional catalogue enrichment and notifications outside the confirmation dependency path when the business permits. The source does not establish a numeric latency budget for this fictional system.

If three required stages take 50, 100 and 200 milliseconds on average, summing gives a rough mean for a simple serial path under matching conditions. Summing their p95 values does not generally produce the end-to-end p95. Correlation, queuing and workload mix affect the distribution. Measure the whole journey and use component timing to diagnose it.

A retry budget also consumes time and capacity. Three thirty-second authorisation attempts cannot fit inside a five-second confirmation promise. Persist uncertainty and return a truthful pending state when the business contract allows it. A timeout value should follow the user promise and recovery design, not a library default alone.

Design review and alternative acceptance

A defensible worked conclusion retains a local order/allocation transaction for the event, isolates payment through durable coordination, and implements bounded admission with a reviewed pending-state message. It postpones separate inventory deployment until a multi-channel ownership need or measured contention limit justifies the new protocol.

A distributed alternative can also pass if it specifies reservation identity, expiry arbitration, duplicate-safe confirmation, payment reconciliation and observability across the boundaries. Reviewers should not deduct points simply because it differs from the worked design. They should challenge unsupported guarantees and unexplained operating effort.

Design a history test before a throughput test

Write a sequential reservation specification: a command with a new ID may reserve one unit if a right remains, a repeated command resolves to its earlier result, and a release can return a held right at most once. Then collect invocations and responses from competing clients. A history is acceptable only if the declared consistency model can explain the observed results while respecting the specification. Linearizability supplies a precise real-time condition for a single concurrent object. [13]

An original test starts with one right and issues two overlapping new reservations. Both success responses are invalid for this specification even if the final database counter is zero. A last-writer-wins counter can hide the lost update while leaving two customers with a promise. Compare the returned

reservation identities and committed obligations, not just a counter after the test. If one response is a timeout, inspect durable state rather than treating it as a failed reservation.

Now issue the same reservation ID twice concurrently. Expect one logical reservation and consistent results under the command contract. Then reuse that ID with a different SKU or quantity: the service should reject an incompatible payload instead of silently treating it as the old command or creating a new reservation. State the key's scope and retention explicitly. These tests address identity semantics that a queue's delivery guarantee cannot decide.

Do not run a thousand identical safe schedules and conclude that every race is covered. Choose barriers at the decision points, inject failures after durable changes and before replies, and retain the shortest counterexample. If the test framework cannot observe a commit or order calls accurately, record that limit. For complex systems, production-grade history analysis and formal modelling may be warranted; the course's small enumerator is a way to learn the logic, not a substitute for those tools.

Exercises

8.1 Why does a duplicate callback test need assertions about inventory and payment state rather than just an HTTP response?

8.2 Using the synthetic table, explain why p95 receipt latency cannot alone establish which policy delivers the best customer outcome.

8.3 Propose a failure experiment for a late payment authorisation after expiry. Include the expected compensation state and an observation that would fail the design.

8.4 Two overlapping reservations both return success for the last unit, yet the final available counter is zero. Explain why a counter-only test misses the defect.

Key points. Correctness tests, workload tests and customer-outcome measures complement each other. Admission policy and pending-state communication are architectural decisions with business consequences.

CHAPTER 9

Data studio brief and semantic contracts

Learning objectives

Define analytical truth at a declared grain and time, separate operational and reporting authority, and design evidence for a published measure.

Beacon must explain its numbers

Beacon now combines records from six operational systems and publishes a daily service dashboard at 07:00 in the business reporting timezone. The assignment volume is ten million new or changed records per day. Three sources provide ordered change positions; two provide daily extracts; one supplies events that may arrive up to seven days late. The business also permits older corrections through an exception process.

Managers want to compare service completion time by division and customer segment. Finance wants a reproducible monthly snapshot. Operations wants recent changes during the day. A single table marked “latest” cannot satisfy all three interpretations without additional contracts. These are distinct reporting products even when they share ingestion infrastructure.

The team proposes a streaming lakehouse and claims that real-time ingestion will make the figures trustworthy. Your task is to evaluate that claim, design two feasible alternatives and specify what a reader can infer from each published version.

Define the question before the pipeline

For the daily service measure, choose a cohort and event boundaries. One possible definition is elapsed time from accepted receipt to first verified closure for requests first closed during the reporting day. That excludes still-open work, so publish backlog age separately. Otherwise a division that leaves difficult cases open may appear faster than one that closes them.

Decide how reopened cases are treated. A “first closure” metric and a “final closure after reopening” metric answer different questions. Neither should be silently substituted for the other after implementation.

Define business-day boundaries, timezone conversion, excluded test records, negative intervals and missing timestamps. Use a versioned metric definition so that a change in meaning can be distinguished from a business trend.

A median across all requests cannot generally be reconstructed by averaging division medians. A mean requires weighted combination using the relevant counts; even then the populations and units must match. A figure that mixes hours and business days is not repaired by a faster query engine.

Grain and identity

A service-event fact has one row per accepted source event identity and source version. A daily request snapshot has one row per request and reporting date under a specified publication version. A completion fact has one row per qualifying completion under the metric policy. These grains are different and should not be joined indiscriminately.

Kimball distinguishes several fact-table forms for different analytical purposes. [14] In this studio, the grain is an explicit contract enforced through keys and quality checks. A completion joined to five status events becomes five rows; summing its duration then weights that request five times. Aggregate each source to the intended comparison grain before combining measures.

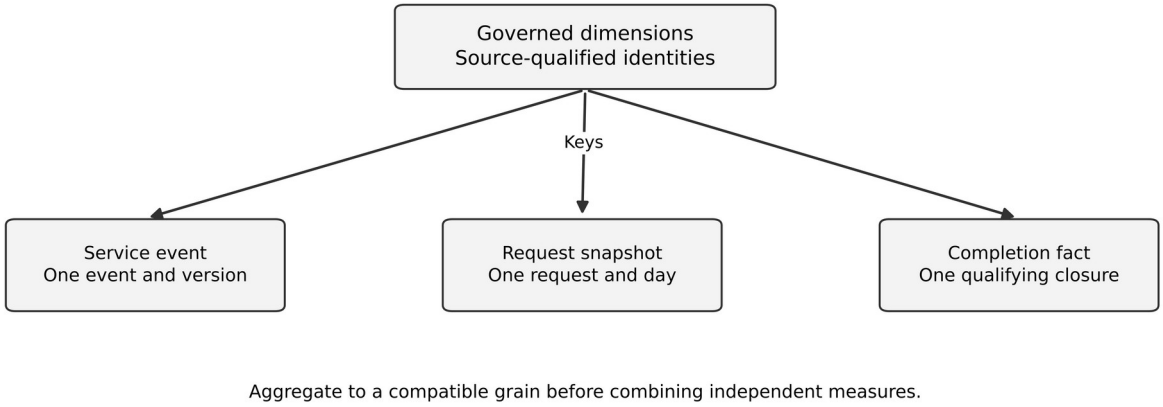


Figure 11 Separate analytical grains connect through governed keys and explicit aggregation

Use source-qualified identifiers. Customer 17 in CRM A is not automatically customer 17 in CRM B. An enterprise customer mapping needs an owner, evidence and treatment of ambiguous matches. Do not merge people or organisations solely because display names resemble one another. The dashboard must make unresolved mappings visible rather than silently inventing certainty.

Three kinds of time

Event time is when the source says the business event occurred. Ingestion time is when Beacon received the record. Publication time is when a particular result became available to users. A corrected source version can change the event-time interpretation after a report has already been published.

For reproducibility, retain enough provenance to answer two questions: what is the currently corrected result for a historical period, and what did the report show at a particular past publication? These are different views. A design may store source versions and materialised publication versions, or reconstruct through controlled history. State the storage and retention implications.

This does not require keeping every personal field forever. Provenance can refer to controlled identifiers, transformation revisions and approved retained evidence. Lifecycle policy may require deletion, restriction or aggregation. The architecture must describe how reproducibility and data obligations interact rather than treating “immutable raw data” as an exemption.

Your first submission

Produce a metric contract, a grain-and-key table, a data ownership map, two pipeline alternatives and a worked example with a duplicate, a late arrival and a corrected event. Draw where publication is gated, where rejected records go, and how a user discovers that yesterday's number changed.

Treat the daily dashboard and monthly snapshot separately. A monthly snapshot may need a formal close and restatement policy; daily reporting may allow a rolling correction window. Both policies need business owners. The assignment deliberately leaves those policy choices open so that your dossier must state and defend them.

Exercises

9.1 Explain how a completion-time dashboard can improve while service quality worsens if unresolved work is excluded.

9.2 A division has ten requests averaging two hours and another has ninety averaging six hours. Calculate the combined mean and explain why averaging the two means equally is wrong.

9.3 Specify the keys and time fields needed to distinguish a duplicate event from a corrected version of that event.

Key points. A data platform serves defined questions through governed meaning, identity and time. Infrastructure can improve delivery, but it cannot infer the intended metric or correction policy.

CHAPTER 10

Pipeline alternatives and versioned publication

Learning objectives

Compare batch and streaming designs, preserve source progress through replay, and make publication an explicit controlled transition.

Design A is a governed batch platform

A batch design obtains consistent extracts or source-supported incremental changes, lands them in restricted storage, validates records and transforms them into curated tables. It computes the daily report under a run identifier and publishes only after required checks pass. The previous valid report remains visible with its timestamp if the new run fails.

At ten million records per day, average arrival volume is about 115.74 records per second. This does not establish the required batch processing rate. If a six-hour nightly window is available, processing ten million records within it requires an average of about 462.96 records per second before accounting for joins, retries and publication checks. Size against the actual window and transformation work.

Batch can be appropriate when the daily deadline and source extract contracts dominate. It may simplify reproducible boundaries and operation for a small team. It still needs correction handling, idempotent reruns, access control and source-to-target reconciliation. A file arriving successfully does not prove it is complete or semantically correct.

Design B separates continuous ingestion from publication

A streaming design ingests supported change streams continuously, records source positions, validates events and updates controlled state. The daily publication can still be a versioned snapshot rather than exposing every intermediate update to managers. This separates freshness of internal state from stability of a published decision product.

Kafka's documentation describes ordering within partitions and qualified transactional processing semantics. [15] A pipeline that also writes an external warehouse must examine that destination's transaction and replay contract. A broker guarantee does not automatically cover a separately committed report table.

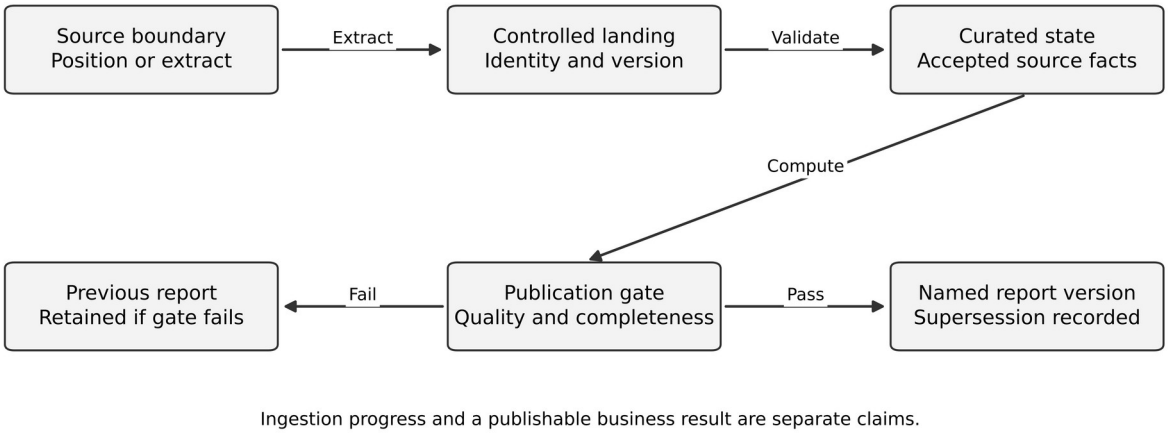


Figure 12 Source progress and quality evidence gate a named publication version

Streaming can reduce latency for the operations product and spread processing across the day. It adds continuous operation, state recovery, schema evolution and late-event policy. It cannot make a once-daily source continuously fresh. Compare the value of earlier information with the extra responsibilities, rather than choosing from volume alone.

Define a replay boundary

For each source, retain the extraction or change position associated with the input. For a snapshot-plus-changes approach, the snapshot and stream must share a documented consistent boundary. An arbitrary file export and a later stream start can leave a gap or double application. The source contract determines the safe procedure.

Transformation must tolerate rerun. For an event with identity E and version 3, replay should not add its measure twice. If a version 2 record arrives later, it should not overwrite version 3 under a monotonic-version contract. If two records have the same identity and version but different content, quarantine the conflict or apply an explicitly approved rule; arrival order alone is not trustworthy evidence.

A deletion marker also needs meaning. It may mean the source entity was deleted, an erroneous event was withdrawn, or a personal-data copy must be removed. Those actions do not necessarily have the same accounting effect. Specify the contract at the source boundary and preserve the approved interpretation in the curated model.

Worked publication history

Suppose event e1 contributes 100 units on day 1, e2 contributes 50 on day 1 but arrives on day 3, and e3 contributes 75 on day 2. The first day-1 publication contains only e1, so its value is 100 and its source-completeness status is provisional. After e2 arrives, the corrected day-1 value is 150. Day 2 remains 75.

Now the source issues e3 version 2 with a corrected value of 70. The currently corrected two-day total is 220. If the platform appends both versions as independent business facts, it incorrectly reports 295. If it overwrites the only copy without publication history, it may be unable to explain why a manager previously saw 225.

One defensible design retains controlled source versions, computes a current accepted version per event, and publishes results with an immutable publication identifier plus a supersession relationship. “Immutable” here describes the report version's identity and recorded result, not an unlimited right to retain underlying personal data. Access and lifecycle policy remain applicable.

Publication failure and partial data

A missing critical source can block publication, permit a visibly partial report or preserve the previous version. The decision belongs to the report owner. Define which sources are mandatory for which measures; one unavailable source need not block every unrelated product. Make completeness machine-readable as well as visible in the interface.

A quality gate should be capable of failing. If every anomaly is automatically downgraded to a warning so that the deadline is met, the gate does not protect the promise. Record the exception, owner, affected

measures and expiry. Repeated exceptions indicate a requirement or source-quality problem that needs a design decision.

Exercises

10.1 Calculate the minimum average processing rate for ten million records in a four-hour window, ignoring overhead. Explain why that is not a final capacity plan.

10.2 For the worked publication history, explain the differences among 100, 150, 225, 220 and the erroneous 295.

10.3 Design an idempotent rerun rule for source identity, version and content. Include the conflicting-same-version case.

Key points. Ingestion, transformation and publication have separate progress and correctness boundaries. A trustworthy platform makes replay and correction observable under an explicit report contract.

CHAPTER 11

Data quality security and analytical evaluation

Learning objectives

Build reconciliation beyond counts, evaluate access through derived data, and judge a data architecture through decisions it can support.

Reconciliation is a set of arguments

Counts detect some failures, but equal counts can hide one missing record and one duplicate. Key-set comparison identifies coverage; version comparison identifies stale records; relationship checks identify orphans; grouped totals identify some mapping defects. Each check has a scope and blind spots. A cryptographic digest can compare canonical representations, but it cannot establish that an approved transformation has the right business meaning.

For Beacon, reconcile each source under its declared position or extract boundary. Compare accepted, quarantined and excluded records so that the accounting closes. If a record is excluded by policy, record the reason and affected measure. Do not make errors disappear by removing them from both the report and the reconciliation denominator.

A quality rule such as “completion time must be non-negative” is structural. A rule such as “closure requires a valid prior receipt” is relational and semantic. A sudden shift in a distribution is a prompt for investigation, not automatic proof of bad data. A real process change can produce unusual values.

An original discrepancy exercise

The supplied local reconciliation experiment starts with three source records and three target records. One target key is missing, another is extra and one retained key has the wrong amount. The counts match, but the key sets and values do not. Run the comparison, repair the target and record which assertions become true.

Extend the experiment with a permitted transformation, such as normalising a status code through an approved mapping. Comparing raw bytes would now reject a correct transformation. The expected result should be generated from the declared mapping, and the mapping itself needs business review. This separates implementation conformance from semantic correctness.

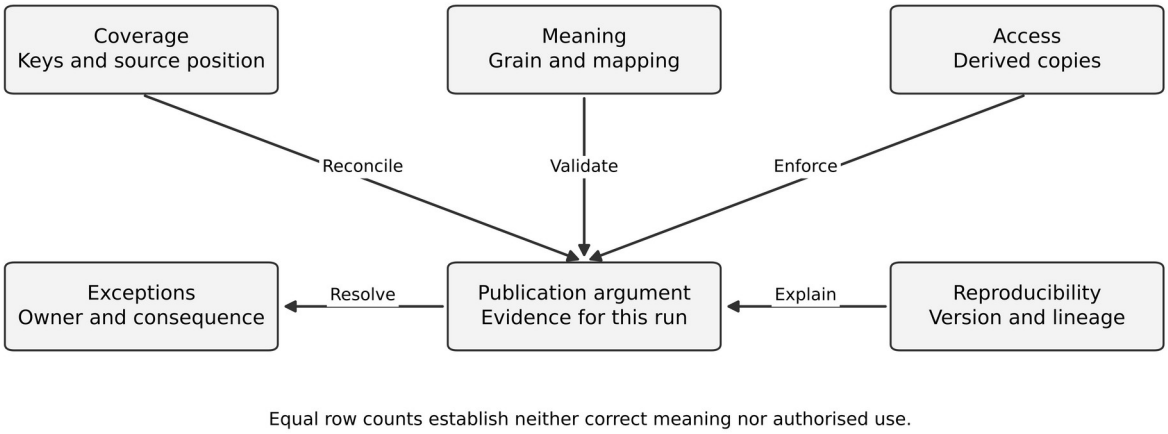


Figure 13 A publication argument combines coverage meaning access and reproducibility

Security follows derived copies

The raw landing area, curated tables, notebooks, dashboard exports and cached results can contain different forms of the same information. Apply access at each relevant boundary. A catalogue label saying “restricted” is not an enforcement mechanism. A dashboard filter can be bypassed if users can query the underlying unrestricted table directly.

Removing names may still leave identifying combinations. A small group with rare attributes can be recognisable, and external datasets may allow linkage. Assess the intended audience and use, and choose aggregation, suppression or other approved controls where needed. Do not promise anonymity solely because an email column is absent.

The GDPR includes purpose limitation, data minimisation, storage limitation and accountability among its principles. [16] This book does not assign a universal retention period. The case architecture needs a policy owner, a data inventory and mechanisms that can implement the approved lifecycle across operational and derived stores.

Deletion and recovery must agree

If a deletion or restriction is approved, determine which copies and derived representations are affected. A later replay or backup restore must not silently reintroduce information contrary to the approved policy. One design records a controlled suppression or lifecycle decision that is reapplied during recovery, without retaining more personal information than necessary for that purpose.

Some aggregate results may be preserved under the applicable policy, while identifiable drill-down records must change. That is a governance decision requiring context, not something the pipeline should infer from file format. Document the effect on reproducibility and explain how users will interpret a historical report whose underlying detail is no longer available.

Evaluate a proposed AI explanation feature

Beacon's sponsor asks for an assistant that explains why completion time increased. A grounded summary can report that a measure rose and point to its definition and source tables. It should not invent a causal explanation from a correlation. A changed case mix, missing source, policy change or genuine delay are competing hypotheses.

Design evaluation around the actual task: factual support, omission of important caveats, correct metric version, access enforcement and handling of absent evidence. Require the same authorised data scope for retrieval and response. A human reviewer should be able to trace consequential statements to approved evidence. NIST's AI RMF provides a voluntary risk-management structure; it does not certify an individual model's accuracy. [17]

The first release could omit automatic causal explanation and provide a controlled metric summary with links to evidence. This is a legitimate scope choice if it meets the business need. A more ambitious feature needs stronger evaluation and operating responsibility, not merely a larger model.

Studio assessment

Submit a data dossier with metric contracts, two pipeline alternatives, a versioned publication example, quality checks, a lifecycle design and an evidence report. Include one research critique on relational independence or analytical grain, distinguishing the source's contribution from your own platform choice. A valid design may be batch, streaming or hybrid when its guarantees fit the reporting products.

Follow an authorisation rule through the analytical architecture

Beacon's dashboard must not disclose a division's restricted records to another division's ordinary analyst. Define the permitted exception separately: a group analyst may see an approved consolidated product. A group role does not automatically permit unrestricted raw-data access. The property concerns the returned information and all reachable paths, not merely whether the dashboard initially loads the correct filter.

Alternative A publishes separated data products. Each division receives a curated product with a narrow identity and its own permitted consumers. Consolidated reporting uses a separately approved transformation and audience. This makes the access boundary visible, but duplicates some processing and creates a need to propagate corrections and lifecycle decisions across products. A copied table with an old access policy is a new risk, even if its values once came from an authorised source.

Alternative B uses a shared curated store with enforced policies. Queries carry independently established principal and resource context. The data service enforces the policy on every supported query path; dashboards and notebooks do not supply their own trusted division claims. Shared views can reduce duplication, but a misconfigured export identity or direct table permission can bypass the intended filter. The policy engine and the store configuration become part of the assurance boundary.

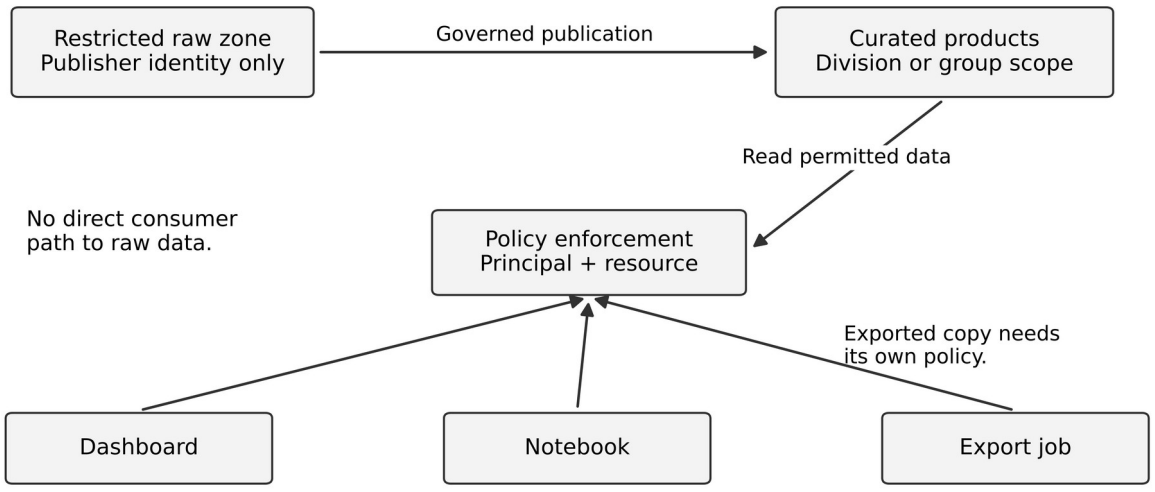


Figure 14 Dashboard notebook and export paths must meet the same data access contract

The figure distinguishes the restricted raw zone from published products. Arrows into a product are governed publication, while consumer arrows show query or export requests. An export is a new copy with its own audience and lifecycle. The diagram therefore does not imply that a row filter continues to govern a CSV after it leaves the service. The case needs a separate rule for approved extracts and their recipients.

NIST's resource-centred access model motivates explicit decisions at resource boundaries rather than trust from network placement. [18] The following test is an original application. Seed six fictional records across two divisions and three tenants. Give one analyst permission to one division, another an approved group reporting role, and a service identity permission only to publish a defined aggregate. Check each permitted and prohibited combination through dashboard, notebook, direct query and export paths. A filter tested only in the dashboard is incomplete evidence.

Record also how policy changes propagate. If an analyst loses access, can a cached result or scheduled export still disclose records? If a job was validly authorised yesterday, must it recheck before running today? The answer depends on whether policy grants a durable approved job or requires current user authority. Make that choice explicit. A token copied into a scheduler should not accidentally decide it.

Run authorisation_matrix.py in the advanced practice files. It checks a deliberately small policy function against an independently specified set of allowed principal-resource pairs and a revoked-delegation case. The program does not implement token validation, a database policy or secure export. Its purpose is to make the business predicates visible enough that a reviewer can challenge them. Reimplementing the same condition in both the function and the expected-results generator would provide weaker evidence than a separate explicit matrix.

Select the shared-store design only if the organisation can demonstrate enforcement on the required paths and operate its policy changes. Select separated products if they better fit ownership and sensitivity, while budgeting the extra lifecycle work. Neither diagram proves non-disclosure under arbitrary compromise; the test and threat model must say which identities, paths and failures are covered.

Exercises

11.1 Source and target each contain one million rows. Name four additional checks and explain one limitation of each.

11.2 A restored raw-data backup includes a record previously removed under an approved policy. Design the recovery control needed before republishing.

11.3 An AI narrative attributes slower service to one division's staff performance. What evidence and alternative explanations must be examined before accepting that claim?

11.4 Choose separated analytical products or a shared policy-enforced store. Specify the same six access cases through dashboard, notebook and export paths, including a revoked user.

Key points. Trustworthy reporting needs coverage, meaning, access and reproducibility together. A fast pipeline and a plausible narrative do not substitute for an auditable analytical contract.

CHAPTER 12

Modernisation studio brief and discovery

Learning objectives

Discover operational dependencies, distinguish hosting change from business change, and design a migration around explicit authority and recoverability.

Orion has a deadline and an incomplete inventory

Orion must move the order-intake capability from an ageing application within nine months because the current hosting agreement ends. The teaching estate contains twelve million historical orders, a relational database, warehouse integrations, scheduled exports and direct reporting queries. The sponsor permits a thirty-minute interruption for the final write cutover, but requires read-only order status to remain available where feasible.

Two teams know the main application. A third team owns a nightly file interface, and an external warehouse partner sends updates through a legacy account. Nobody has confirmed every writer to the order tables. The target can represent new order states that the legacy schema cannot represent directly. This last fact materially limits rollback after new target writes.

The migration must preserve accepted orders and avoid duplicate fulfilment. The business can postpone optional new states until after stabilisation if the architect shows why that restriction reduces risk. Your assignment is to compare a direct replacement, a phased capability migration and a temporary hosting move that buys time.

Start with observed authority

Inventory every path that can create or change an order: application APIs, batch jobs, database accounts, file imports, support scripts and partner integrations. Interview owners, inspect configuration and observe representative runtime activity. A dependency seen only at month end can be absent from a one-week trace. Absence of evidence from a narrow window is not evidence of absence.

Separate readers from writers and classify the consequences of failure. A reporting extract can tolerate a different delay from a warehouse dispatch update. A direct database writer may bypass the new routing layer entirely. Until those paths are governed, a diagram showing “all traffic through gateway” is an intention rather than the current architecture.

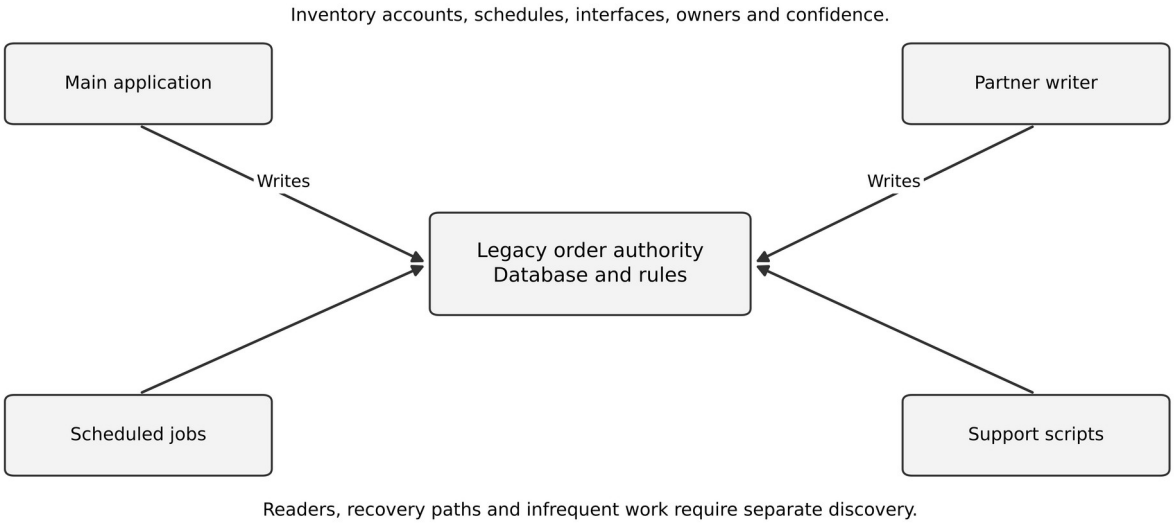


Figure 15 A migration inventory must include hidden writers and operational dependencies beyond the main user interface

Maintain a confidence field in the inventory. “Confirmed from configuration and observed execution” is stronger than “owner believes unused”. Assign an investigation and deadline for consequential unknowns. Do not colour every application green because it has a named owner if the authority boundary remains unverified.

Compare the three strategies

A direct replacement can minimise the duration of coexistence but concentrates data, interface, process and operating change at one point. It requires strong discovery and rehearsal evidence. A phased capability migration reduces the size of each transition but introduces routing, replication and temporary operating states. A temporary hosting move may reduce deadline risk while preserving technical debt and creating another later migration.

Microsoft's strangler pattern describes incremental replacement through a routing boundary. [19] In Orion, the important design question is which capability or cohort can move while preserving a single write authority. Splitting by page URL is insufficient if both old and new paths update the same business object through uncontrolled writers.

A defensible interim choice may move read-only status first, retain legacy order writes, then transfer a bounded order cohort after the write fence is verified. Another valid answer may rehost temporarily because the nine-month deadline is incompatible with safe semantic replacement. Make the cost of that delay visible rather than treating it as failure to modernise.

Model transition states as real services

State A keeps legacy writes and builds a target shadow projection. State B serves selected reads from the target while legacy remains authoritative. State C transfers write authority for an explicitly defined domain or cohort after fencing and reconciliation. State D retires the replaced responsibility after evidence of stable operation and removed dependencies.

Each state needs a support owner, deployment path, access policy, data flow, monitoring and recovery procedure. Coexistence is not merely an arrow between “current” and “target”. It may last months and become the most complicated operating state in the programme.

The cohort rule must be stable for in-flight work. If customer group A moves today, define what happens to orders created yesterday and still being fulfilled. A policy that routes each request according to the customer's latest group can send successive updates to different authorities. Persist ownership at an appropriate business boundary or implement a governed transfer.

Your first submission

Create a current-state dependency map with confidence levels, three strategy options, four transition states and a write-authority table. Identify the earliest reversible step and the first step after which recovery must preserve new business state. Include a plan to discover the external partner's write path before cutover.

Use the nine-month hosting deadline as a constraint to analyse, not a reason to assert that every task fits. If the evidence does not support replacement in time, propose a feasible fallback with cost, consequence and a decision deadline.

Exercises

12.1 Explain why a week of network observations may miss a critical migration dependency.

12.2 Compare a temporary rehost and a phased replacement under the nine-month deadline. State what evidence would favour each.

12.3 Design a cohort rule that keeps an in-flight order under one write authority during migration.

Key points. Discovery and intermediate states are architectural work. Migration strategy must fit both the business deadline and the evidence available about authority, semantics and recovery.

CHAPTER 13

Data movement write fencing and recovery

Learning objectives

Connect source snapshots to change streams, define a complete write fence, and distinguish traffic reversal from recovery after new business transactions.

Preserve a consistent data boundary

An initial copy must represent a defined source state. Continuing changes must start from a position that correctly follows that state under the source's supported mechanism. Otherwise the migration can miss updates between the copy and stream start, or apply changes twice without a safe duplicate rule.

Record source position, schema revision, transformation revision and extraction scope. Track inserts, updates and deletions. A low average replication lag can hide a stalled table or partition, so monitor the lag and errors of each relevant stream. The final cutover checks must account for the exact source boundary, not merely report that the average lag was small an hour earlier.

A source transaction that modifies an order and its lines must not become a published target state that exposes the order without required lines. The target can stage partial application and publish a consistent boundary, or use an appropriate transaction mechanism. The migration contract must say when a target record becomes safe for readers.

Map meaning rather than columns

Suppose legacy status X means cancelled for web orders but archived for older telephone orders. A mapping that always changes X to cancelled destroys meaning even when every row copies successfully. Profile by process and historical period, involve business owners, and preserve the rule version used to transform each relevant record.

Some transformations are lossy. Splitting a free-text address cannot invent a missing country reliably. Converting a date into a timestamp cannot recover an unrecorded time. Quarantine ambiguous cases, preserve source evidence under policy and define manual or conservative treatment. Do not improve apparent completeness by fabricating values.

A write fence must cover all writers

A maintenance page blocks one user route. It does not stop scheduled jobs, partner accounts or direct database scripts. A valid fence prevents every relevant source writer from committing new authoritative changes beyond the cutover boundary. The implementation may use application controls, account restrictions, database mechanisms or a combination, but the claim must be tested across all paths.

After fencing, let in-flight work reach the agreed boundary or terminate it under a defined policy. Capture the final source position, apply remaining changes and reconcile critical data. Only then permit target writes for the migrated scope. Retain evidence of which authority owned each interval.

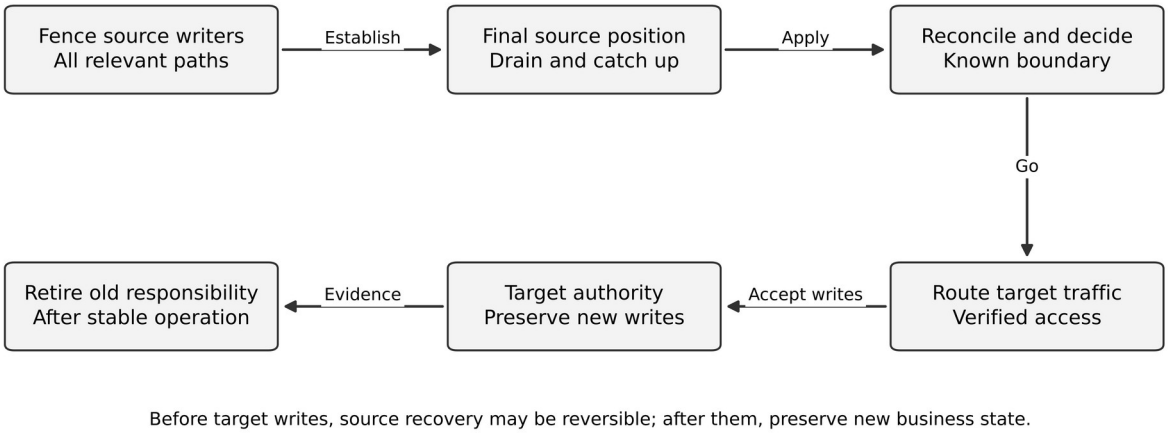


Figure 16 The final source position and verified write fence separate source authority from target authority

If the target must call a legacy warehouse function after cutover, define whether that function changes the migrated order state or only owns warehouse execution. A residual dependency is not necessarily invalid, but it must not reintroduce uncontrolled dual authority. Use explicit commands and accountable outcomes rather than arbitrary writes into both databases.

Rollback before and after the authority switch

Before target writes, reverting routing can be feasible if legacy remains intact and current. After target writes, a simple route change can hide orders that exist only in the target. A database restore may remove them. Recovery must therefore preserve and reconcile new business state, even if that makes recovery slower than switching a load balancer.

One option temporarily restricts target features to states representable in both systems and maintains a tested reverse path. Another commits to forward recovery, repairing the target while preserving its authority. A third uses a controlled freeze and transformation back to legacy when the business accepts the interruption. Each requires an explicit limit and rehearsal.

The fact that target-only states exist means a generic reverse transform may be impossible. Do not promise rollback until the business agrees either to postpone those states or to a forward-recovery approach. This is a concrete example of how product scope affects reversibility.

A recovery dossier fragment

Decision M-07 permits target-only order states only after a two-week stabilisation period. Before that point, the target writes a compatibility subset and retains a controlled change history. Test M-T12 creates new target orders, simulates an application failure and rehearses the approved recovery path without duplicate warehouse commands. The evidence records elapsed time, reconciliation results and unresolved exceptions.

This does not make every possible rollback safe. It establishes a bounded capability for the tested schema, operation set and failure scenario. A later schema expansion invalidates the assumption and should trigger a new review. Recovery readiness is a maintained property of the design, not a one-time checkbox.

Exercises

13.1 Explain how a snapshot and an unrelated later CDC start position can lose data despite both processes completing successfully.

13.2 Write a cutover fence checklist for the application, batch jobs, partner integration and support scripts. State the evidence that each writer is stopped.

13.3 The target has accepted 300 orders in a state legacy cannot represent. Compare forward recovery with a return to legacy, identifying the missing business decisions.

Key points. Migration correctness depends on a consistent source boundary, governed semantic mapping and one write authority. Recovery after target writes is a business-state problem as well as an infrastructure problem.

CHAPTER 14

Cutover rehearsal and modernisation review

Learning objectives

Evaluate a timed cutover plan, define decision deadlines with recovery allowance, and close a migration through operational and business evidence.

Measure the critical sequence

A fictional rehearsal records five sequential activities: establishing the write fence takes four minutes, draining and applying remaining changes takes nine, critical reconciliation takes six, routing and smoke checks take three, and the final go/no-go decision takes two. The total is twenty-four minutes within the thirty-minute interruption window.

The six-minute remainder is not automatically sufficient recovery allowance. If reversing the pre-cutover state takes eight minutes, a decision at minute twenty-four already makes the thirty-minute promise infeasible. Establish the latest safe decision point from the recovery path as well as the forward path. Some validation may happen before the interruption; some tasks may run concurrently only if their dependencies permit it.

A second rehearsal increases change application by eight minutes. The total becomes thirty-two minutes before variation or recovery. The plan fails the stated window. Adding a “contingency” line in a document does not change the arithmetic; reduce and rehearse the work, change the mechanism, negotiate the window or postpone cutover.

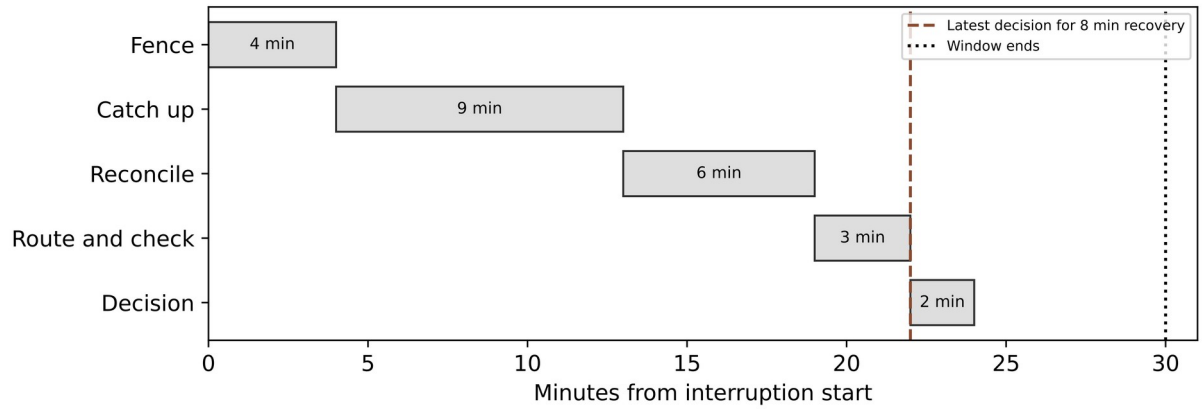


Figure 17 A cutover window includes both forward progress and a feasible decision point for recovery

Test the readiness argument

Readiness includes data reconciliation, target capacity, correct service identities, operator access, DNS or routing behaviour, monitoring, support procedures and customer communication. A successful import under a developer administrator account does not establish the production service account's permissions. A successful database restore does not establish that the complete business service is usable.

Use representative data volume and shape. Twelve million nearly identical test rows may fail to reveal long text, old encodings, unusual relationships or a few very large customer accounts. Include the exceptions discovered during profiling and report which historical classes remain untested.

Rehearse failure at consequential points: fence established but catch-up incomplete; reconciliation mismatch; routing changed but target access broken; target has accepted new orders; warehouse acknowledgment lost. The response should follow the authority state reached, not a universal “switch back” instruction.

A change card during the review

Two days before cutover, the warehouse partner reports an undocumented batch writer that updates order status after midnight. The owner says it can be stopped manually, but there is no test evidence. The planned cutover is at 23:45.

This discovery is a release-blocking uncertainty if the writer can violate the fence or create competing authority. A defensible decision delays the cutover until the writer is controlled and rehearsed, or changes the scope and timing with evidence that the risk is removed. A manager's confidence does not substitute for the missing mechanism.

The architect should present the consequence plainly: the current plan may allow post-fence source writes, so the reconciled boundary can become stale before target authority begins. Offer a specific closure task: identify the account and schedule, disable or redirect it through the approved process, verify denied writes in rehearsal, and update recovery procedures. This is more useful than a generic warning that migration is risky.

Retire the old responsibility deliberately

After stable target operation, retirement requires evidence that users, integrations, reports, jobs and support procedures no longer depend on the replaced function. Remove access and scheduled work through controlled changes. Cancel licences or resources only when their responsibilities are actually removed, and retain records according to policy.

Measure the promised benefits. If infrastructure cost falls but two teams still reconcile data manually every day, the transition may have shifted cost rather than removed it. If release lead time improves but incidents rise, investigate the change in operating conditions. A technical cutover is a milestone; realised business improvement is a separate outcome.

Compare the modernisation alternatives again

The direct replacement may now be less credible because discovery remains incomplete. A phased approach can move read-only status while the writer inventory is resolved. A temporary hosting extension or rehost may be justified if it prevents a forced unsafe cutover, but its cost and later exit work must be included.

A good review may therefore change the original recommendation. Preserve the decision history and explain which new fact changed the ranking. The goal is not to defend the initial architecture at any cost; it is to make the current decision reasonable under current evidence.

Studio assessment

Submit a migration dossier with the dependency inventory, transition architectures, authority table, reconciliation rules, timed rehearsal, recovery decision points and retirement evidence. Include an explicit response to the undocumented writer. A strong solution states what must be true before each transition and how the organisation remains operable if that condition is not met.

A write fence must be enforced at the destination

In the Orion cutover, a pause in the user interface does not establish that every writer has stopped. A batch process, support tool or delayed queue consumer can still issue an update. An original fencing design assigns an increasing authority epoch to the active writer. The destination checks that epoch atomically with the write and rejects obsolete epochs. Merely attaching an epoch to a log entry without enforcing it cannot prevent an old writer from changing state.

Suppose epoch 7 belongs to the legacy system. Cutover grants epoch 8 to the target and changes the destination's accepted authority. A delayed legacy command with epoch 7 arrives afterward. It must be rejected even if its business timestamp appears newer. The rejection should be observable and routed for reconciliation where appropriate. A token accepted on one destination but ignored by another leaves a gap in the overall argument.

Compare that mechanism with a simpler operational fence: remove the legacy write permission and network path, stop its jobs, and verify denied writes before enabling the target. This can be sufficient if every writer is known and the enforcement really covers them. It may be easier to operate than adding epochs to a legacy platform that cannot validate them. The tradeoff is the strength of the writer inventory and the ability to prove the old authority has been removed.

Neither technique resolves semantic rollback. If the new application creates a state the old application cannot represent, restoring old write authority can still lose meaning. A rollback decision needs both a functioning fence and a valid representation or compensating business plan for every new state. The latest safe decision time in the rehearsal must reserve enough time to execute and verify that plan. A green infrastructure dashboard is insufficient evidence that the business can safely resume.

Exercises

14.1 A thirty-minute window requires eight minutes to restore the pre-cutover state. What is the latest decision point if that recovery must finish inside the window? Explain why the twenty-four-minute plan is insufficient for this claim.

14.2 The target contains twelve million rows, matching the source. Propose a release gate that can detect incorrect mappings and missing relationships despite equal counts.

14.3 Draft a short decision notice for the newly discovered midnight writer, including the specific closure evidence required.

14.4 A legacy writer logs epoch 7 after epoch 8 becomes active. What must enforce the fence, and why does this not establish semantic rollback?

Key points. Cutover feasibility includes forward work, uncertainty and recovery. A migration is complete only when the new service operates correctly and the replaced responsibility is actually retired.

CHAPTER 15

Architecture review communication and judgement

Learning objectives

Run a focused design review, communicate uncertainty to different audiences, and revise an architecture without losing decision history.

Review a bounded decision

A useful review names the decision, the architecture version and the evidence available. “Review the platform” is too broad for one meeting. “Decide whether the first release can proceed with CRM B's unresolved-outcome procedure” gives the participants a concrete responsibility. Invite the people who own the relevant business promise, implementation, operation and risk.

Distribute a short decision brief before the review. State the problem, mandatory conditions, viable alternatives, recommendation, key evidence and unresolved issues. Attach detailed diagrams and experiment records as supporting material. A long presentation should not consume the time needed to challenge assumptions.

The review can conclude proceed, proceed with conditions, investigate further or reject. Conditions need an owner and observable closure evidence. “Security to review” is an activity, not a closure criterion. “Cross-tenant download, search and export tests pass for the release configuration, with policy-owner approval of exceptions” is reviewable.

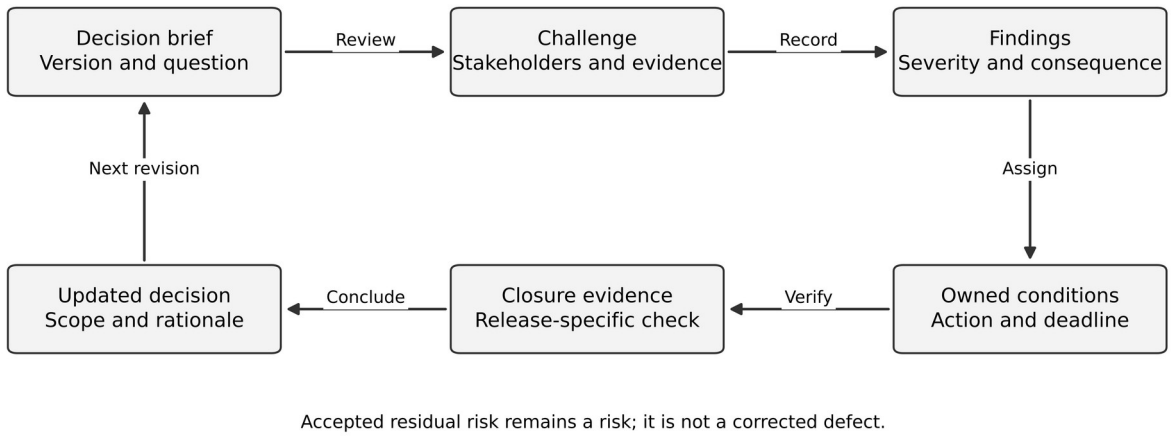


Figure 18 Review findings lead to owned conditions and a versioned decision rather than disappearing into meeting notes

Separate findings from preferences

A finding should identify the affected requirement, the exact defect or uncertainty, the consequence and the evidence. A critical issue could permit loss of accepted transactions or unauthorised disclosure. A major issue may leave an important recovery claim unsupported. A minor issue might be an ambiguous diagram label. An optional enhancement should not be disguised as a blocker.

If a reviewer prefers another database, ask which requirement the preference improves and what evidence supports that improvement. If the proposed design violates an invariant, do not downgrade the issue because the team likes the current stack. The same evidence discipline applies to both the author and the reviewer.

Avoid false precision in risk matrices. Multiplying subjective likelihood and impact numbers can create a useful discussion aid, but it does not make the resulting score a measured probability or monetary loss. Record the rationale and examine whether a different plausible judgement changes the priority.

Communicate the same decision at two levels

For executives, explain the business consequence, choices, cost range, uncertainty and decision needed. For engineers, explain ownership, contracts, failure states, configuration and test evidence. These should be two views of the same argument. Do not present an unconditional business promise while placing its unresolved conditions only in a technical appendix.

For example, the integration recommendation can say: “The current quota cannot meet the proposed thirty-minute recovery target. We can buy additional capacity, reduce admitted demand or agree a longer target. Accepted requests remain durable in each option.” The technical appendix then shows the backlog equation, quota scope and proposed limiter. The main message remains understandable without hiding the mechanism.

A diagram should support the spoken explanation. Start with the business journey, then show the boundary responsible for the key guarantee. Use the failure sequence when discussing resilience. A deployment diagram with many product icons may be less useful than a small sequence that exposes the unresolved outcome.

Practise disagreement without inventing certainty

When challenged, restate the concern and identify whether it is a new fact, a different preference or a different interpretation of the requirement. A new provider limit may change feasibility. A preference for lower cost may change the comparison weights. A different meaning of “confirmed” may reveal that the design solves the wrong problem.

If you do not know, state the uncertainty and propose the smallest investigation that could resolve it. Do not fill the gap with a confident product claim. Equally, do not defer every ordinary design choice indefinitely. A reversible choice with low consequence may proceed under a recorded assumption while a high-impact uncertainty blocks release.

Record learning after the review

Keep a finding register with status and evidence links. When a condition closes, record what was tested and which design version it applies to. A configuration change can invalidate previous evidence. If a finding is accepted as residual risk, name the responsible business owner and review date rather than marking it “fixed”.

Compare the revised design with the original. Which assumptions changed? Which decisions remained robust? Did the new evidence justify the cost of investigation? This reflection develops judgement across cases and prepares the move to enterprise-level decisions in Book 3.

Exercises

15.1 Rewrite “the architecture is scalable and secure” as a review conclusion with scope, evidence and two unresolved conditions.

15.2 A senior reviewer insists on microservices without identifying a requirement. Draft a respectful response that brings the discussion back to evidence.

15.3 Distinguish a closed finding, an accepted residual risk and a deferred investigation using one studio example for each.

Key points. A review should expose consequential assumptions and produce a decision whose conditions can be checked. Clear communication preserves the same truth across business and technical audiences.

CHAPTER 16

Portfolio assessment and professional transfer

Learning objectives

Assemble evidence of architectural reasoning across domains, assess alternatives consistently, and plan supervised experience beyond the course.

The final portfolio

Select your strongest two studio dossiers for detailed presentation and include concise comparative reports for the other two. Retain the original briefs, assumptions, diagrams, ADRs, experiment records, findings and revisions. The purpose is to show how you reasoned and improved the design, not merely to display a finished diagram.

Each detailed dossier should include a two-page decision brief; a requirement and assumption register; connected context, runtime, data, sequence and deployment views; at least three consequential ADRs; a failure and recovery design; security and lifecycle reasoning; a cost model with explicit fictional inputs; and an evidence report. Add a statement of what remains unverified.

The comparative reports should explain which principles transfer and which constraints change the answer. Durable intent appears in integration, commerce and migration, but its authority and external effects differ. A publication version in analytics solves a different problem from an order's payment state. Transfer the reasoning, not the surface diagram.

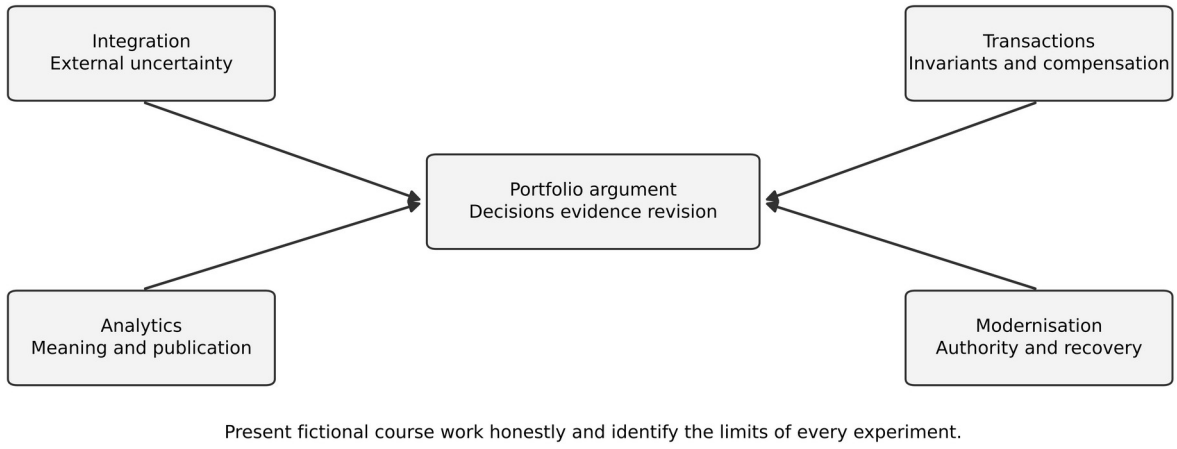


Figure 19 Four studios contribute different evidence to one architecture portfolio

A common assessment rubric

Use the following rubric for each dossier. The points organise feedback; they are not an industry certification or a claim of measured professional competence. Award credit for a defensible alternative even when it differs from the worked design.

Dimension	Points	Evidence expected
Problem and requirements	15	Clear outcomes, constraints, scenarios and owners
Boundaries and invariants	20	Coherent authority, transactions, states and data contracts
Alternatives and judgement	15	Viable competing options and explicit tradeoffs
Failure security and recovery	20	Threats, uncertainty, compensation and tested recovery claims

Dimension	Points	Evidence expected
Evidence and research use	15	Reproducible method, honest limits and accurate source transfer
Operation economics and transition	10	Feasible support, cost assumptions and operable intermediate states
Communication and revision	5	Clear dossier and traceable response to challenge

A strong submission earns at least 75 points under this teaching rubric and has no unresolved critical flaw hidden behind a score. A design that can knowingly oversell, lose accepted work or expose another tenant's data must address that defect regardless of its visual presentation. If a case constraint cannot be met, a clearly argued infeasibility conclusion can demonstrate strong judgement.

The oral defence

Prepare a ten-minute explanation followed by twenty minutes of questions. Spend the opening on the business problem and promise. Explain two decisions that most strongly shape the result, show one difficult failure sequence, and end with the evidence and conditions for proceeding. Keep the full dossier available for detailed questions.

Use a change card during the questions: a vendor quota halves; the payment outcome window expires earlier; a source starts correcting old events; a legacy writer remains active; or the operating team loses a key skill. You have five minutes to identify affected assumptions and propose a response. You do not need to redesign every component immediately. You do need to show which guarantees are now at risk.

The examiner should distinguish reasoning quality from speed. A careful answer that identifies an unresolved contract is stronger than an instant answer that invents one. Ask what observation would make you change your recommendation. If no possible evidence could change it, the decision may be a preference rather than an evaluated conclusion.

A research synthesis

Write a 1,500-word synthesis using at least four primary sources already cited in the course. Compare the kind of knowledge each provides: a formal result, a design argument, a research method, a standard or operational guidance. Explain one case decision informed by each source and one claim that the source does not establish.

Do not reproduce long source passages or substitute a bibliography for analysis. The reader should see your transfer reasoning. For example, a consensus paper can clarify the replicated-state boundary while leaving payment compensation outside it. A modularity paper can inform a code boundary while leaving deployment topology undecided. A product manual can establish a documented feature while leaving its suitability for the case open.

From course work to role preparation

Present the portfolio honestly as fictional course work. Explain the experiments you ran and the limits of their models. Do not imply that an invented case was a client deployment or that an assistant-led review was independent peer review. Your existing degrees and programming experience provide useful context, while real architecture responsibility also requires working with stakeholders and operating constraints.

Choose a next experience that closes a specific gap: participate in a design review, help maintain an ADR set, trace a real incident across dependencies, assist with a migration rehearsal or review an integration contract with an experienced colleague. Record what you learned and how it changes a previous assumption. These experiences complement the books rather than being replaced by them.

Book 3 moves from one solution to the organisation's portfolio, capabilities, decision rights and transformation programme. Carry forward the habit of making assumptions and evidence visible. Enterprise scope changes the scale and the stakeholders; it does not remove the need for precise reasoning.

Exercises

16.1 Compare the role of durable state in two studios. Explain one shared principle and one important difference in authority.

16.2 Apply the rubric to your first and revised dossier. Identify the evidence behind the largest improvement rather than merely reporting a higher score.

16.3 Write a ninety-day development plan with one technical, one stakeholder and one operational objective, each with observable evidence of progress.

Key points. A portfolio should demonstrate decisions, evidence and learning across contexts. The course prepares you to contribute to architecture work; continued practice and feedback develop that preparation into professional judgement.

Exercise answers and assessment guidance

Attempt each question before using this guidance. Numerical answers use the stated simplifying assumptions. Design answers illustrate defensible approaches and criteria for judging alternatives; they are not the only valid architectures.

Chapter 1

1.1 One scenario measures receipt continuity: during a thirty-minute CRM outage at the declared arrival rate, every eligible request that receives a successful receipt remains durably accountable. A second measures recovery: after the CRM returns under a stated quota and ongoing arrival rate, the oldest pending handoff falls below an agreed age within a defined interval. The first observes local acceptance and persistence; the second observes downstream completion and backlog. Include eligibility rules, failure scope and the party responsible for the promise.

1.2 State that the option is currently ineligible because the required export cannot be demonstrated. Offer three explicit choices: obtain and test a sufficient export mechanism, choose an eligible alternative, or ask the authorised business owner to reconsider the requirement with the exit consequences visible. Do not allow an overall score to imply compliance. If the requirement changes, record the new decision and reassess the options under that contract.

1.3 The context view identifies the payment provider as an external authority; the runtime view locates the order coordinator; the sequence shows timeout after possible authorisation; the data view preserves operation identity and unknown outcome; the deployment or operations view supplies durable recovery and reconciliation access. A contradiction would be a sequence that retries with the same operation identity while the data model stores no identity, or an executive promise of cancellation while the state model still has unresolved authorisation.

Chapter 2

2.1 Net drain is 28 - 12 = 16 operations per second. Time is 24,000/16 = 1,500 seconds, or twenty-five minutes. It fails the twenty-minute objective even before variability, retries or competing quota use. A valid response increases successful capacity, reduces admitted arrivals, changes the recovery promise or changes the workflow; adding an unbounded queue does not reduce the drain time.

2.2 First, the independent constant-probability binomial calculation gives a one-sided 95 percent upper failure-probability bound of about 0.2991 percent, which does not establish a 0.01 percent failure level. Second, the trials may not represent production conditions or independent failures, and may omit relevant outage scenarios. Zero observed failures is evidence about a finite sample, not a universal reliability guarantee. Distinguish statistical uncertainty from missing scenario coverage.

2.3 Vary only the worker policy while holding build, workload, source data, provider quota and failure schedule fixed. Measure distinct completed business operations, unresolved outcomes, oldest pending age, attempts and throttling. Inject a worker crash after remote success and a bounded provider outage. Predeclare a rule such as: both policies must preserve accepted work and avoid prohibited duplicates; among eligible policies, prefer the one meeting the recovery target with lower operating cost. Repeat to investigate variation rather than selecting one favourable run.

Chapter 3

3.1 CRM A's contract enforces uniqueness of the external reference and returns the existing effect. CRM B offers neither uniqueness nor necessarily immediate lookup visibility. The same client retry can therefore create a second remote record. A common adapter interface can expose confirmed, rejected and unresolved outcomes, but it cannot manufacture the missing provider guarantee. Require reconciliation or an accepted bounded manual process for B, or negotiate a stronger contract.

3.2 Persist the selected division and destination with the accepted receipt under a server-controlled routing rule. Continue the in-flight request under that authority unless an explicit transfer process changes it. A transfer must coordinate the original and new case ownership and prevent duplicate creation. Routing every retry from the customer's latest division is unsafe because one logical request can cross authorities without a governed transition.

3.3 Logical isolation can use separate work classes, quotas, concurrency limits and fair selection within one deployment. Separate deployments can limit process crashes, memory exhaustion or release impact, but may still share a database and vendor quota. A scenario where one adapter repeatedly exhausts process memory may justify a separate process or deployment. A shared database outage is not solved by that boundary alone. Credit the design for the failure domain it actually isolates.

Chapter 4

4.1 The expired worker can still send its remote request after another worker has claimed the intent. The local lease prevents some competing local updates but does not stop an external provider from accepting both calls. CRM A's unique-reference contract makes repeated creation for the same operation return one effect under its stated guarantee. Both workers must reuse that reference; a new reference on retry defeats the protection.

4.2 Move the operation into a controlled resolution state, stop new automatic creation attempts and retain both candidate references with evidence. An authorised operator or domain process decides which record corresponds to the intended request and how any duplicate is resolved. The system must not claim confirmed uniqueness, safe deletion or known rejection merely because two search results exist. Late responses from old attempts must be reconciled against the recorded resolution.

4.3 A valid ADR chooses polling for a small number of delivery paths if measured database load and fairness meet the objective, while acknowledging work-selection implementation and replay limits. The broker alternative supports independent consumers and buffering but adds publication checkpoints and operating responsibilities. A review trigger could be a measured inability to meet per-division age targets under an agreed peak, or an approved independent consumer requiring replay. The trigger should identify a requirement, not a preferred product.

Chapter 5

5.1 The required net drain is 14,400/1,800 = 8 operations per second. Adding ongoing arrivals of 6 gives at least 14 successful handoffs per second in the ideal model. The call quota must also allow failed attempts, status lookups and other consumers. Fourteen calls per second is therefore not automatically sufficient for fourteen successful handoffs per second.

5.2 Attempts may include duplicates and failed or retried work; the aggregate may hide a starving division; and completions may be measured at the broker or worker rather than the remote business effect. Also inspect ongoing arrival rate, oldest pending age and unresolved outcomes. A high rate can coexist with a growing backlog if arrivals exceed successful departures.

5.3 One response negotiates a higher shared quota and implements account-scoped rate control, retaining durable receipts while showing delayed handoff status. Until the upgrade is verified, publish the longer recovery expectation and provide an owned worklist for time-sensitive cases. Another response limits new admissions with an explicit message before acceptance. Silently discarding accepted work is invalid. Explain who can approve the changed customer promise and how support tracks affected receipts.

Chapter 6

6.1 After reservation, A = 1,997, R = 3 and C = 0. Confirming two gives A = 1,997, R = 1 and C = 2. Releasing the remaining unit gives A = 1,998, R = 0 and C = 2. The sum remains 2,000 and no quantity is negative. The partial-confirmation policy was explicitly permitted; a system with all-or-nothing baskets would need a different transition contract.

6.2 The hot product still receives 350 attempts per second at its allocation boundary. Dividing other products across ten groups does not divide that one invariant. Measure the actual lock, update or coordinator bottleneck. Possible responses include bounded admission, an efficient atomic reservation operation or controlled sub-allocations, each with its own correctness and fairness implications. More partitions are not by themselves evidence of increased capacity for one key.

6.3 The event should identify the order, event version, tenant or seller scope, committed reservation references, the verified payment-authorisation reference required by policy and the confirmation time. Its

meaning is that the declared confirmation conditions have been met, not that a checkout request entered a queue. Consumers need duplicate handling and a rule for later cancellation or refund events; those later events do not erase the fact that confirmation previously occurred.

Chapter 7

7.1 If expiry commits first, it moves the active reservation to expired and returns its allowance. Confirmation against that reservation must fail or enter a new-resolution path; it must not commit the already released units. If confirmation commits first, it moves the reservation to committed and expiry must have no releasing effect. Both operations test the expected active state or version at the same authority. The order's state must agree with the winning transition.

7.2 Holding until payment is known reduces the risk of late authorisation without stock, but can lock inventory behind abandoned or slow attempts. Bound holds, authenticate or rate-limit admission as appropriate and monitor age. Fixed expiry frees inventory predictably but requires late-result reconciliation and possible payment compensation. Keep unresolved orders visible and define the fresh-reservation or reversal policy. Neither option eliminates the need to know the provider's eventual outcome.

7.3 The total of spendable allowances, committed sales and allowance in transfer must not exceed the original allocation under the simplified model. Remove or fence the source's spendable allowance before making it independently spendable at the destination, using a durable transfer identity and duplicate-safe transitions. If the destination result is unknown, retain a pending transfer and query or retry that same operation safely. Do not restore source allowance merely because a response timed out.

Chapter 8

8.1 A callback can return HTTP success while applying the business effect twice. Assertions must show that the same payment event does not decrement inventory again, create another charge or emit an inconsistent order transition. Check terminal state, operation identity and the accounting invariant. Also verify that a valid duplicate receives a response compatible with the provider's retry contract so that harmless repeats do not become an endless delivery loop.

8.2 Receipt latency measures only one part of the journey. The policies admit different populations and leave different unresolved counts. Examine rejected-customer behaviour, time to final outcome, distinct valid orders, allocation correctness and why confirmation counts differ. The synthetic data cannot establish user satisfaction or production throughput. A defensible conclusion states the additional evidence needed rather than choosing the lowest latency automatically.

8.3 Reserve one unit, let payment authorisation succeed remotely while hiding its response, expire and release the reservation, then deliver the late authorisation. Require the system to avoid confirming against the expired reservation. It may obtain a fresh reservation under policy or enter payment reversal with a durable compensation identity. Inject reversal failure and require a visible cancellation-pending state. Marking cancellation complete without verified reversal, overselling or issuing a new charge fails the design.

Chapter 9

9.1 A closed-case cohort excludes unresolved work. If difficult requests remain open, the completed subset can become faster even while backlog and customer delay grow. Publish the closure-time measure together with backlog count and age, and define the cohort explicitly. A change in case mix or closure policy can also alter the number. The metric should support investigation rather than claim staff performance directly from one average.

9.2 Total elapsed hours are $10 \times 2 + 90 \times 6 = 560$ across 100 requests, giving 5.6 hours. Equal averaging gives 4 hours and incorrectly weights the small division as heavily as the large one. The weighted calculation is valid only if the requests belong to compatible populations and use the same duration definition and units. For medians, even weighting division medians does not generally reconstruct the overall median.

9.3 Retain source identity, source-qualified event ID, source version, event time, ingestion time and a content digest or comparable canonical representation. A repeat of the same identity, version and content is a duplicate. A higher version under a monotonic-version contract is a correction. The same identity and version with different content is a conflict requiring investigation. Publication identity separately records which accepted versions contributed to a report.

Chapter 10

10.1 Four hours contain 14,400 seconds. Ten million divided by 14,400 is approximately 694.44 records per second. This is an ideal minimum average before transformation cost, skew, joins, retries, quality checks and publication. It also assumes the full four-hour interval is available. Measure representative processing and reserve time for validation and recovery before declaring the window feasible.

10.2 The first provisional day-1 report is 100 because e2 has not arrived. The corrected day-1 result is 150. Before the correction to e3, the accepted two-day total is 225. After e3 changes from 75 to 70, the corrected total is 220. The erroneous 295 adds both e3 versions as separate business events. Preserve publication history so that a prior 225 and a current 220 can both be explained accurately.

10.3 Use a source-qualified event key and accept the highest supported version under the source contract. Repeating the same version and identical content has no additional effect; an older version cannot replace a newer one. A conflicting same-version record is quarantined or resolved under an explicit owner-approved policy. Recompute affected published measures through a versioned correction process. Do not silently choose whichever record arrived last.

Chapter 11

11.1 Compare distinct key sets for coverage, source versions for freshness, foreign-key relationships for orphans and grouped business totals for mapping errors. Key sets do not verify values; versions rely on correct source semantics; relationship checks can pass with wrong but valid references; totals can hide offsetting errors. Add selected record comparisons and semantic review where consequential. All checks must share a consistent extraction boundary and account for approved exclusions.

11.2 Before republishing, reapply the current approved lifecycle decisions to the restored data and derived outputs, then verify that restricted or removed records do not reappear. Preserve only the controlled evidence needed to perform that function under policy. Reconcile downstream caches, exports and indexes as applicable. A backup restoration procedure that ignores post-backup deletion decisions is incomplete even if the database restore itself succeeds.

11.3 Verify the metric definition, source completeness, publication version, case mix, reopened cases, late events and policy changes. Compare plausible operational causes and seek evidence that distinguishes them. Correlation with a division does not establish that staff performance caused the delay. A generated narrative should state supported observations and uncertainty, with traceable evidence and human review for consequential interpretation.

Chapter 12

12.1 Monthly or quarterly jobs, dormant recovery procedures, rare partner updates and infrequent support scripts may not run during the observation window. Combine runtime evidence with configuration, schedules, account inventories and owner interviews. Record the confidence and remaining gaps. A narrow trace is useful evidence of observed dependencies, not a complete proof that every writer has been found.

12.2 A temporary rehost is favoured if the hosting deadline is firm and safe semantic replacement cannot be demonstrated in time, provided the old application can operate securely and reliably in the temporary environment. Phased replacement is favoured when a stable routing and authority boundary exists and intermediate states can be supported. Compare total coexistence, migration and later exit costs. The deadline alone does not make an unverified replacement feasible.

12.3 Assign ownership at order creation or through an explicit migration registry, and persist that assignment for the order's lifecycle. Move an in-flight order only through a governed transfer that establishes its state and new authority. Do not route each update according to a customer attribute that can change independently. The rule must also cover partner and batch writers, not only browser traffic.

Chapter 13

13.1 Changes committed after the snapshot's represented state but before the CDC start can be absent from both inputs. Starting too early can also duplicate effects unless replay is safe. Use a source-supported consistent snapshot and change position, and verify its semantics. Completion of both tools only establishes that each did its own work, not that their boundaries compose correctly.

13.2 Identify each writer's account, route, schedule and owner; apply the approved stop, deny or redirect control; wait for in-flight work to reach the defined boundary; and attempt representative writes to verify they cannot commit to the old authority. Capture the final source position only after the fence is effective.

Record exceptions and recovery steps. A maintenance banner proves only that one user interface has changed.

13.3 Forward recovery preserves target authority and the 300 orders while correcting the defect, but requires a safe operating or containment mode. Returning to legacy requires a business-approved representation or transformation of those states, reconciliation and prevention of duplicate fulfilment. If legacy cannot represent them, a simple rollback is not available. The business must decide whether to postpone the new states in advance or accept a forward-recovery commitment.

Chapter 14

14.1 The latest decision point is minute 22 if an eight-minute recovery must finish by minute 30. A forward plan that decides only at minute 24 leaves six minutes and cannot support that recovery claim. Move relevant checks earlier, shorten and rehearse recovery, redesign dependencies or change the approved window. The calculation assumes deterministic durations; real variation requires further allowance.

14.2 Require matching governed key coverage, relationship integrity, source change position, approved transformed values and grouped business totals, with quarantined exceptions resolved or explicitly accepted. Include tests of authorisation and the business journey because data equality alone does not establish service readiness. Validate mapping semantics with owners; a technically consistent wrong mapping can pass byte-level transport checks.

14.3 A suitable notice states that cutover is paused because the newly identified writer can invalidate the final source boundary after the planned fence. Closure requires identification of its account and schedule, an approved control preventing or redirecting its writes, a rehearsal demonstrating the control, and updated recovery instructions. Name the owner and the next decision time. Avoid describing an untested manual promise as completed mitigation.

Chapter 15

15.1 A bounded conclusion might approve the receipt design for the tested peak and configuration, citing local commit, duplicate-key and tenant-scope evidence, while requiring verification of the vendor's uncertain-outcome contract and a timed restore before production release. It should identify the architecture version and owner for each condition. It does not claim unlimited scale or universal security.

15.2 A constructive response asks which required quality or team responsibility the service split would improve, then proposes comparing the alternatives using that scenario. Acknowledge possible benefits such as independent ownership or scaling while also evaluating distributed failure and operating cost. Do not dismiss the reviewer or accept the preference as evidence. Record the decision and the observation that would justify revisiting it.

15.3 A closed finding has evidence that the defect is corrected, such as a passing release-specific cross-tenant access test. An accepted residual risk remains, such as bounded manual CRM B reconciliation approved by the accountable owner. A deferred investigation has not established the fact, such as an untested provider export path with a deadline before procurement. Their statuses must not be merged into a generic green indicator.

Chapter 16

16.1 Integration and commerce both preserve an operation identity and durable intent so that a crash does not erase responsibility. Integration's local receipt does not establish a CRM effect; commerce's confirmed order additionally depends on reservation and payment policy. Analytics uses durable publication identity to explain a past result, which is a different authority question. A strong comparison names the invariant and the external boundary in each case.

16.2 Use the same rubric on both versions and cite specific evidence: a new failure sequence exposes and resolves a race; a revised contract makes pending state honest; a repeated experiment verifies a recovery assumption. Do not award points merely for extra pages or more citations. Explain remaining weaknesses and whether the revision changes the release decision. Self-assessment is formative feedback, not external certification.

16.3 One plan implements and explains a duplicate-safe local workflow in month one, participates in or simulates a stakeholder design review in month two, and completes a supervised recovery or incident-analysis exercise in month three. Evidence could include reviewed code, a revised ADR with feedback and a timed recovery report. Tailor the objectives to actual opportunities and preserve honesty about simulated versus real operational experience.

Second edition advanced exercises

4.4 CRM may have committed a case whose search index has not caught up, or it may never have committed. The negative lagging search distinguishes neither. Retain the original request and attempt identity in an uncertainty state. Seek authoritative lookup, an adequately scoped destination uniqueness contract or controlled reconciliation. Do not blindly retry with a new identity. A manual process is acceptable only if it follows the same evidence rule and has an owner.

7.4 Across both regions, available rights plus active reservations plus committed sales plus rights in transit must equal 2,000 under the fixed-allowance assumptions. Removing 100 into transit makes them unavailable to North. South may have credited them despite the lost reply; reclaiming and spending them in North could duplicate the right. Recover using the transfer ID and authoritative outcome. The answer must cover replay, recovery from an old snapshot and prevention of obsolete writers.

7.5 Reservation-first may block scarce stock while payment is pending; payment-first can produce a payment authorisation without stock. A late verified payment cannot revive an expired reservation without a new valid stock decision. In either order, compensation can remain uncertain and needs a durable state, safe provider-specific retry or reconciliation, and an owner. Prefer the option whose admitted intermediate states fit the business policy and measured workload; do not award full credit for an unqualified universal ordering.

8.4 Two successful promises violate the sequential one-right specification. A lost update may leave the final counter looking plausible while two reservation records or external confirmations exist. Check the operation history, distinct reservation obligations and conservation together. Under a valid one-unit model, at most one distinct new reservation can succeed before release or replenishment. A timeout is not equivalent to a definite failed operation.

11.4 A defensible matrix includes own-division permitted data, another division denied data, approved group aggregate, unapproved group raw detail, revoked access and an export with an unauthorised audience. Apply these cases to each actual path; a nonexistent path should be inaccessible, not silently untested. Accept either architecture with consistent authority, lifecycle and policy-freshness rules. A correct dashboard filter alongside unrestricted notebook access fails the stated property.

14.4 The destination or another unavoidable authoritative write boundary must reject an obsolete epoch atomically with the attempted change. Logging alone cannot fence. An operational removal of all old write permissions is a possible alternative when demonstrably complete. Even perfect fencing does not make target-only business states representable in legacy; rollback still needs validated mappings, reconciliation and an acceptable treatment of new commitments.

Experiments templates and research tasks

The companion support folder contains four small Python experiments and editable Markdown templates. Use Python 3.10 or newer and run each file with python3 followed by its filename, or use the equivalent interpreter command on your system. The scripts use only the standard library, operate locally and do not contact a cloud service. They are teaching models with assertions, not production reference implementations.

Experiment 1 Recovery and sample uncertainty

Run recovery_and_evidence.py. It calculates four integration recovery scenarios and the binomial zero-failure bound from Chapter 2. Expected drain times include 1,200 seconds for 24,000 items at net twenty per second, 1,500 seconds at net sixteen, and 7,200 seconds for 14,400 items at net two. Change the arrival rate to equal service rate and explain why there is no finite drain time. State which production observations would replace the constant-rate assumptions.

Experiment 2 Reservation event ordering

Run reservation_schedules.py. It exercises confirmation before expiry, expiry before confirmation, duplicate confirmation and late payment after expiry. The assertions check the simplified allocation invariant and permitted terminal states. The model is in memory and single-threaded; it enumerates event orderings rather than proving database concurrency. Extend it with a partial quantity or failed compensation and write the missing invariant before changing the implementation.

Experiment 3 Publication versions

Run publication_versions.py. It prints the provisional, corrected and superseded totals from the data studio and rejects conflicting content for the same event version. Extend the input with an older version arriving after a newer one, then verify that the accepted current result does not regress. Explain how a production implementation would persist run identity, source position, access policy and lifecycle decisions.

Experiment 4 Migration reconciliation

Run migration_reconciliation.py. It reports a missing key, an extra key and a changed amount despite equal row counts. It then checks a repaired target. Add a parent-child relationship and an approved status transformation. State why the script cannot by itself approve a cutover: it does not test source consistency, write fencing, operator access, elapsed time or business acceptance.

Reusable dossier files

The templates cover the decision brief, requirement and assumption register, ADR, experiment record, review findings and migration gate. Replace instructional prompts with your own case analysis and keep evidence identifiers stable. A completed template should explain a decision; filling every field with generic language is not sufficient.

For each studio, write one 400–600 word research critique. Choose a relevant primary source, identify its supported contribution, then explain one transfer and one limit for the case. The final synthesis in Chapter 16 compares these forms of evidence. Source reading time is included in the eight-week study estimate.

References

References are numbered in order of first citation. Online sources were accessed on 4 October 2026. Product documentation should be checked again before implementation.

[1] S. Brown, "The C4 model for visualising software architecture," Accessed: Oct. 4, 2026. [Online]. Available: https://c4model.com/

[2] M. Nygard, "Documenting architecture decisions," Nov. 15, 2011. Accessed: Oct. 4, 2026. [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[3] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for architecture evaluation," Carnegie Mellon University, Software Engineering Institute, CMU/SEI-2000-TR-004, Aug. 2000. Accessed: Oct. 4, 2026. [Online]. Available: https://www.sei.cmu.edu/documents/629/2000_005_001_13706.pdf

[4] J. D. C. Little, "A proof for the queuing formula L = lambda W," Operations Research, vol. 9, no. 3, pp. 383-387, 1961, doi: 10.1287/opre.9.3.383. Accessed: Oct. 4, 2026. [Online]. Available: https://pubsonline.informs.org/doi/abs/10.1287/opre.9.3.383

[5] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," Communications of the ACM, vol. 15, no. 12, pp. 1053-1058, Dec. 1972, doi: 10.1145/361598.361623. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.lafayette.edu/~gexia/cs301/resources/parnas.html

[6] Amazon Web Services, "Transactional outbox pattern," AWS Prescriptive Guidance. Accessed: Oct. 4, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[7] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," RFC 9110, Jun. 2022. Accessed: Oct. 4, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[8] RabbitMQ, "Consumer acknowledgements and publisher confirms," Accessed: Oct. 4, 2026. [Online]. Available: https://www.rabbitmq.com/docs/confirms

[9] Microsoft, "Azure for AWS professionals," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/

[10] PostgreSQL Global Development Group, "Transaction isolation," PostgreSQL 18 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://www.postgresql.org/docs/18/transaction-iso.html

[11] H. Garcia-Molina and K. Salem, "Sagas," in Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, 1987, pp. 249-259. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf

[12] J. Dean and L. A. Barroso, "The tail at scale," Communications of the ACM, vol. 56, no. 2, pp. 74-80, Feb. 2013, doi: 10.1145/2408776.2408794. Accessed: Oct. 4, 2026. [Online]. Available: https://barroso.org/publications/TheTailAtScale.pdf

[13] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," ACM Transactions on Programming Languages and Systems, vol. 12, no. 3, pp. 463-492, Jul. 1990. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.cmu.edu/~wing/publications/HerlihyWing90.pdf

[14] R. Kimball, "Fact tables," Nov. 5, 2008. Accessed: Oct. 4, 2026. [Online]. Available: https://www.kimballgroup.com/2008/11/fact-tables/

[15] Apache Software Foundation, "Design," Apache Kafka 4.1 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://kafka.apache.org/41/design/design/

[16] European Parliament and Council of the European Union, "Regulation EU 2016/679," Official Journal of the European Union, L 119, pp. 1-88, May 4, 2016. Accessed: Oct. 4, 2026. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679

[17] E. Tabassi, "Artificial Intelligence Risk Management Framework AI RMF 1.0," NIST AI 100-1, Jan. 2023. Accessed: Oct. 4, 2026. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[18] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST SP 800-207, Aug. 2020. Accessed: Oct. 4, 2026. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-207

[19] Microsoft, "Strangler fig pattern," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig